pg-probe

POSTGRESQL CAPTURE REPORT

db-prod-01

Saturday, 2026-10-24 · 14:57:24 to 15:10:14 UTC

Duration
13 min
155 samples, every 5 s

Status
Complete
The capture ran to the end.

Sources
51 of 51 sources
Collected, 1 of them in part.
See page 2.

Host

System	Debian GNU/Linux 13 (trixie)
Kernel	Linux 7.2.8-9-omarchy, x86_64
CPUs	32
Memory	62.7 GiB

PostgreSQL

Version	17.11 (Debian 17.11-1.pgdg13+2), primary
Cluster size	583.0 MiB
Connected as	postgres, superuser

Capture

Tool	pg-probe 0.2.2
Endpoint	127.0.0.1:5432
Charts cover	The whole session

Data coverage

pg-probe read 51 sources. 50 collected in full. Charts show the rest as unknown, never as zero.

Collected in part

Some of the data came in. The rest is expected to be missing.

Source	Why
`postgres.extensions.visit`	These databases do not accept connections: template0, template1

Contents

System

1. CPU utilization by state (%)
2. Load average, 1 / 5 / 15 minutes
3. Memory by use, up to MemTotal (GiB)
4. Swap used (GiB)
5. Disk throughput, all disks (MiB/s)
6. Disk busy time, five busiest disks (%)
7. Network throughput, all interfaces (MiB/s)
8. Pressure stall, some tasks, 10 s average (%)
9. Filesystem usage
10. Processes running and blocked
11. Swapped in and out (MiB/s)
12. Major page faults per second
13. Pages scanned for reclaim per second
14. Packets dropped and errors per second, all interfaces
15. Average read latency, five busiest disks (ms)
16. Average write latency, five busiest disks (ms)
17. Average requests in flight, five busiest disks
18. Memory and swap charged to the cgroup (GiB)
19. Cgroup I/O, all devices (MiB/s)
20. Cgroup limit events per interval

PostgreSQL

1. Client connections by state
2. Cache hit ratio per interval (%)
3. Transactions per second
4. Temporary files written (MiB/s)
5. WAL and buffer writes (MiB/s)
6. Database size
7. Largest tables
8. Dead tuples
9. Lock requests waiting, by lock type
10. Longest lock wait (s)
11. Locks held, by mode
12. Longest time in state since state_change (min)
13. Active backends, by wait event type
14. Deadlocks, recovery conflicts and checksum failures per interval
15. Sessions ended abnormally per interval
16. Time reading and writing blocks (ms per second)
17. Queries canceled by recovery conflicts per interval
18. WAL files archived per interval
19. SLRU blocks read from disk per second
20. I/O by operation, all backends (MiB/s)

Queries

1. Rows per second

Overview

Each value is the last sample, or the last interval for rates. Peaks cover the whole session.

CPU busy
2.8%
Peak 35.8% at 15:06

Memory used
30.8%
19.3 of 62.7 GiB
Peak 39.2% at 15:01

Disk busy
2.3%
dm-0, the busiest disk
Peak 100.0% at 15:01

Load, 1 minute
3.26
32 CPUs online
Peak 11.06 at 15:08

Connections
2
of 200 max_connections
Peak 65 at 15:02

Cache hit ratio
100.0%
Lowest 17.4% at 15:08

WAL generated
0 B/s
Peak 12.6 MiB/s at 15:07

Replica lag
None
No standby connected

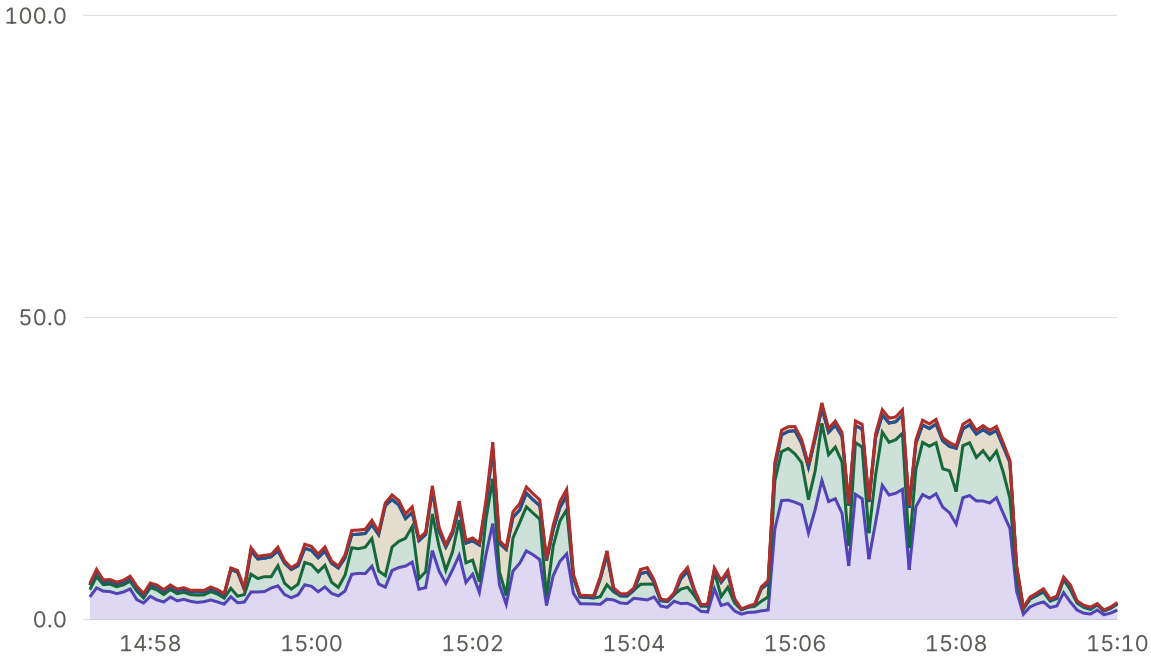
System

CPU utilization by state (%)

Share of CPU time in each state, averaged over all CPUs. User is time in programs such as PostgreSQL backends, and System is time in the kernel. I/O wait is idle time while a disk request was pending. Steal is time the hypervisor gave to other virtual machines.

What to look for

Add User and System to get busy time. When busy time stays above 80% for minutes, check Load average. Load above the CPU count at the same time can mean queries waited for a core, but Linux load also counts tasks waiting on disk. A p95 near 100% with a low p50 in the table means short bursts, not a constant shortage. I/O wait above 10% means CPUs sat idle while disk requests were pending, so check the latency charts. Steal above a few percent means the instance shares its CPUs with busy neighbors.



CPU utilization by state (%): summary

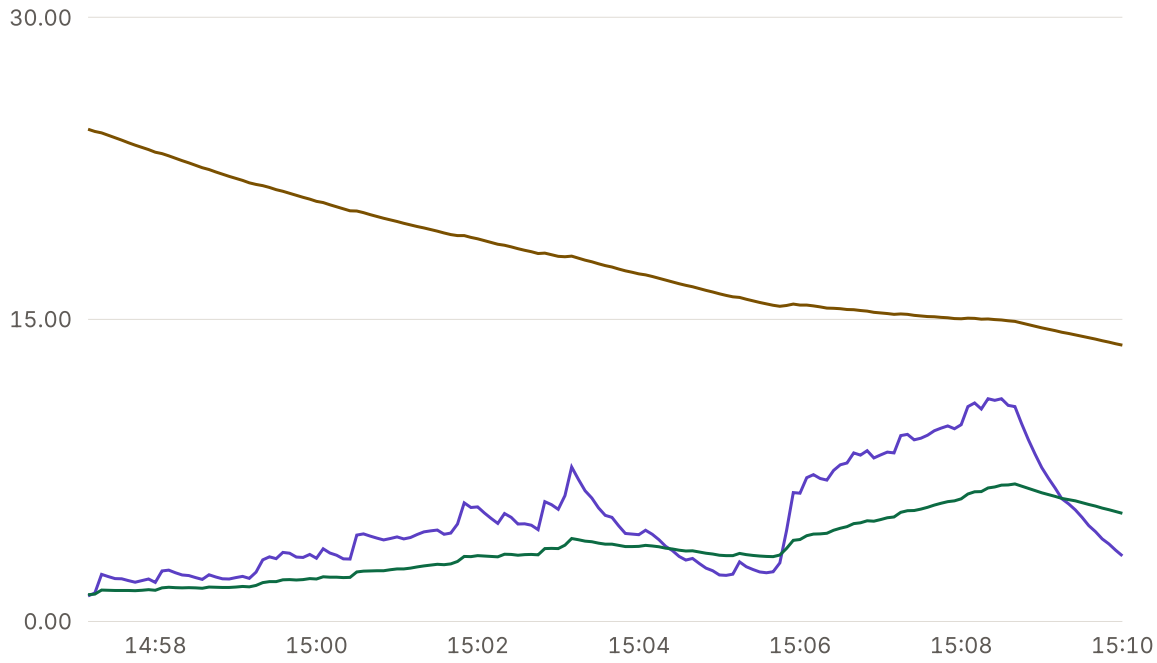
Series	p50	p75	p95	p99	Current
1. User	4.6	10.7	20.6	21.9	1.6
2. System	2.3	5.9	8.6	9.1	1.0
3. I/O wait	2.3	3.3	6.2	7.7	0.1
4. Steal	0.0	0.0	0.0	0.0	0.0
5. Nice, IRQ, softirq	0.4	0.7	0.9	1.0	0.2

Load average, 1 / 5 / 15 minutes

Average number of tasks running or waiting to run, over the last 1, 5 and 15 minutes. On Linux the count also includes tasks waiting on disk.

What to look for

Compare the load with the number of CPUs on the Load tile of the Overview. Load above the CPU count means tasks waited for a core or a disk. A 1 minute line far above the 15 minute line is a spike that just started. High load with low CPU busy time means the tasks wait on disk, not on CPU.



Load average, 1 / 5 / 15 minutes: summary

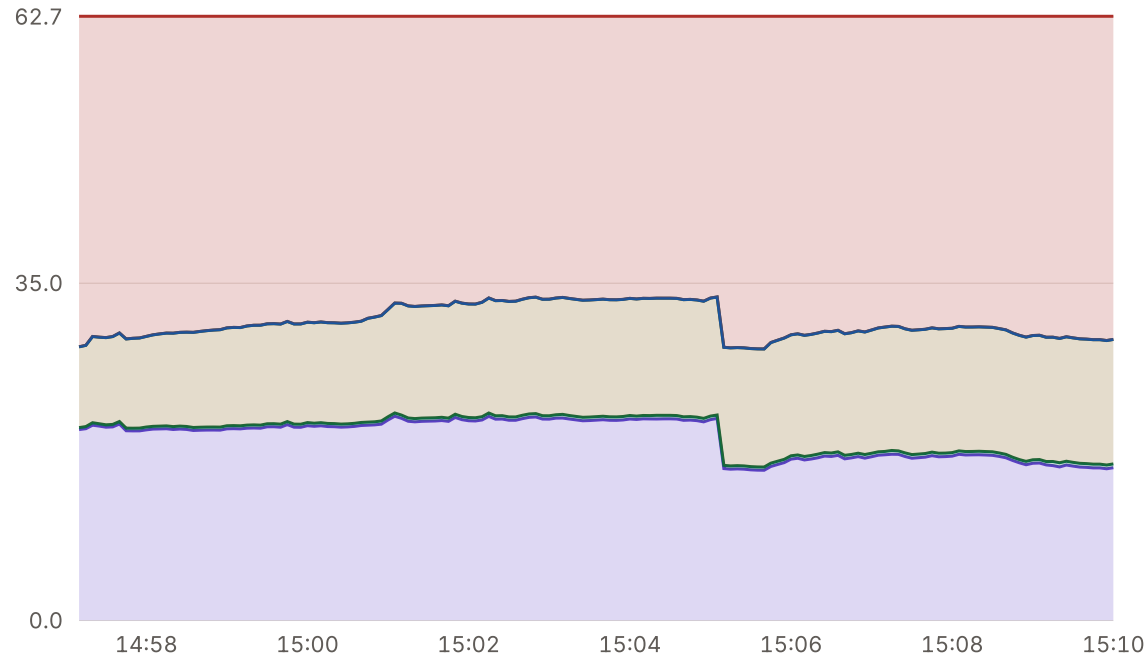
Series	p50	p75	p95	p99	Current
1. 1 minute	4.37	6.82	10.04	11.02	3.26
2. 5 minutes	3.35	4.94	6.46	6.77	5.37
3. 15 minutes	17.67	20.44	23.57	24.29	13.72

Memory by use, up to MemTotal (GiB)

How the host used its RAM. Used is memory held by programs. Shared memory includes the PostgreSQL shared_buffers. Page cache is file data the kernel keeps for fast reads, and Buffers is filesystem metadata. Free is memory nobody uses. With huge pages, shared_buffers counts as Used instead.

What to look for

Low Free alone is normal, because Linux fills spare memory with page cache. The warning sign is a page cache that shrinks while Used grows. PostgreSQL then finds less of its data in memory and reads more from disk. Read this chart with Cache hit ratio and Swap used.



Memory by use, up to MemTotal (GiB): summary

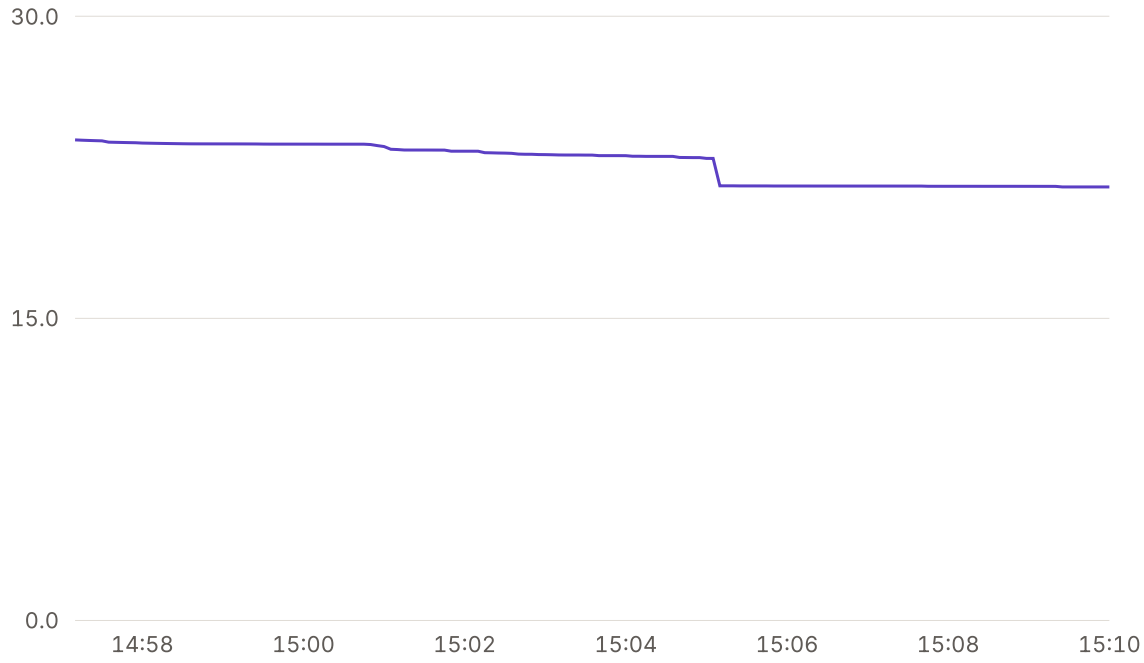
Series	p50	p75	p95	p99	Current
1. Used	19.9	20.7	21.0	21.1	15.9
2. Shared memory	0.3	0.4	0.4	0.4	0.4
3. Page cache	12.1	12.7	12.9	12.9	12.9
4. Buffers	0.0	0.0	0.0	0.0	0.0
5. Free	32.2	33.0	34.2	34.5	33.5

Swap used (GiB)

Swap space in use on the host.

What to look for

A flat line is usually old data the kernel moved out long ago, and it does no harm. A line that rises during the session means the host ran short of memory while the capture ran. Swapped in and out shows whether the host reads that data back, which is what slows queries.



Swap used (GiB): summary

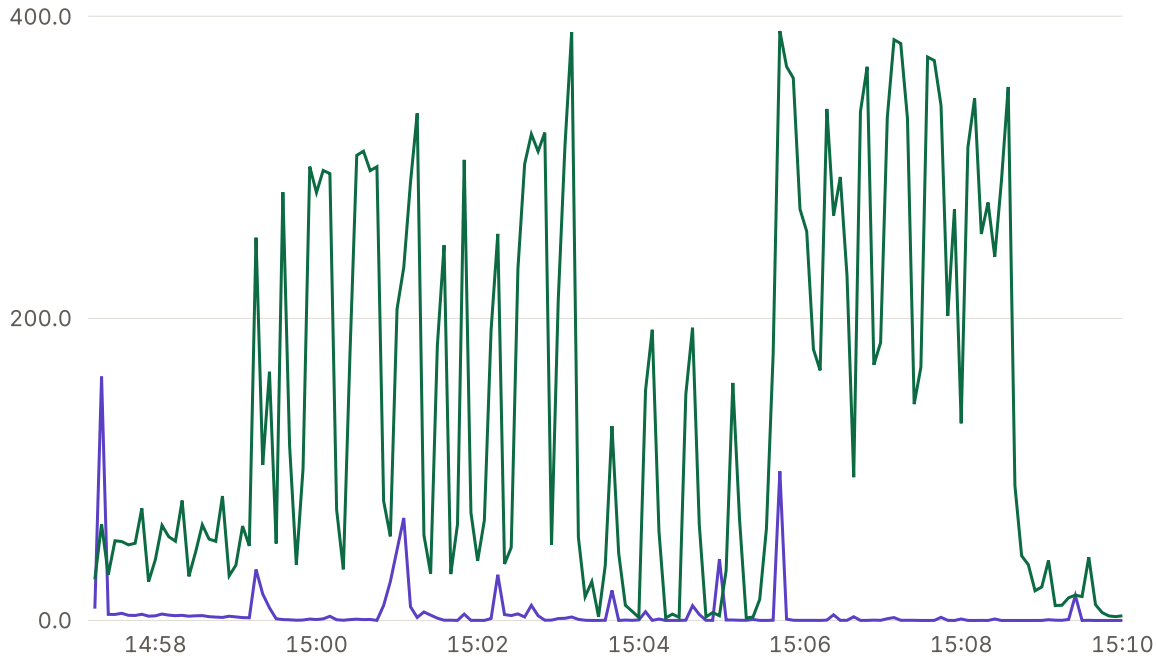
Series	p50	p75	p95	p99	Current
1. Used	23.1	23.6	23.7	23.8	21.5

Disk throughput, all disks (MiB/s)

Bytes read from and written to all disks per second.

What to look for

A line that stays flat at the same top value for minutes is often a limit of the device or the cloud volume. A read spike that lines up with a drop in Cache hit ratio means queries read data that was not in memory. Write spikes every few minutes are often checkpoints. Compare with Disk busy time and the latency charts.



Disk throughput, all disks (MiB/s): summary

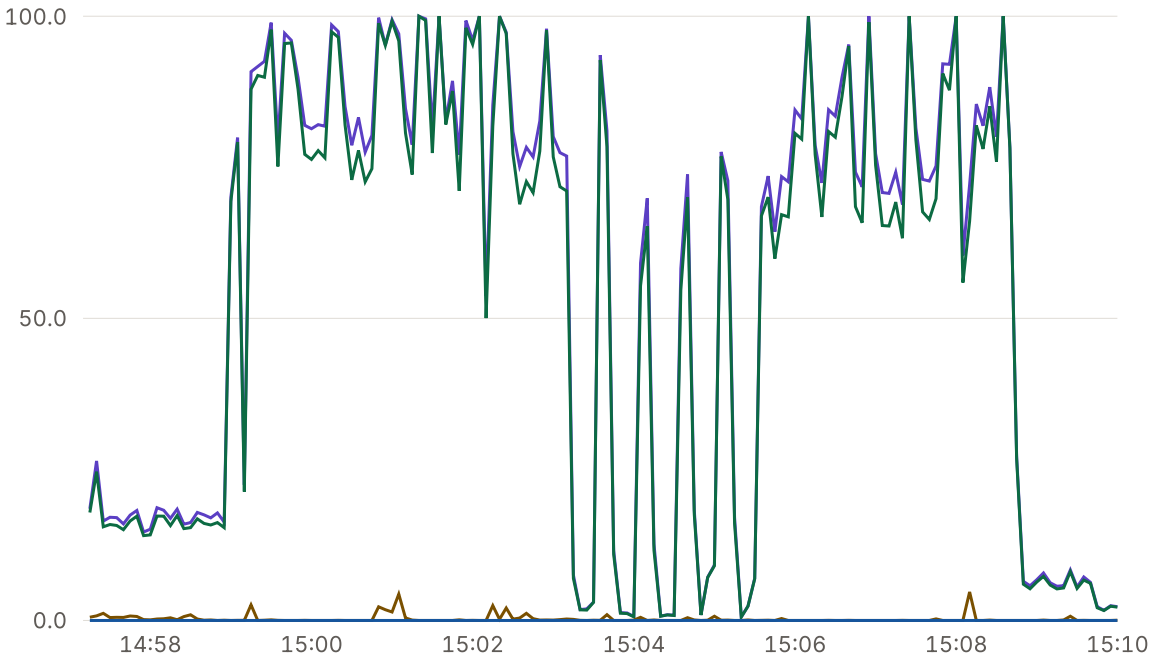
Series	p50	p75	p95	p99	Current
1. Read	0.5	3.1	21.9	82.4	0.0
2. Write	79.2	271.2	361.7	386.8	3.1

Disk busy time, five busiest disks (%)

Share of time each of the five busiest disks had at least one request in flight.

What to look for

On a single spinning disk, values near 100% mean the disk is saturated. SSDs, RAID arrays and cloud volumes serve many requests at once, so they can show 100% and still have room. For those, trust read and write latency more than this chart.



Disk busy time, five busiest disks (%): summary

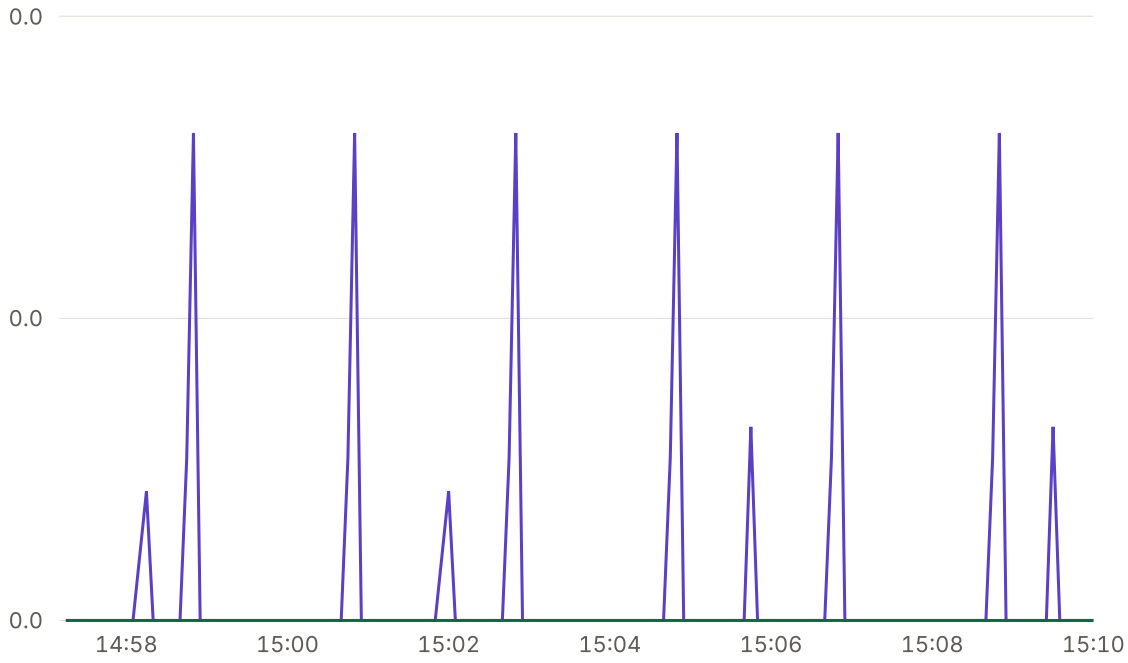
Series	p50	p75	p95	p99	Current
1. dm-0	73.5	84.5	100.0	100.0	2.3
2. nvme0n1	69.1	80.9	99.5	100.0	2.2
3. zram0	0.0	0.1	1.3	3.4	0.0
4. nvme1n1	0.0	0.0	0.0	0.0	0.0

Network throughput, all interfaces (MiB/s)

Bytes received and sent per second, summed over all interfaces.

What to look for

A flat top near the link speed means the network is the limit. A 1 Gbit/s link tops out at about 119 MiB/s. Sudden spikes often match backups, a standby catching up or queries that return large results. Sent traffic on a primary grows with each standby.



Network throughput, all interfaces (MiB/s): summary

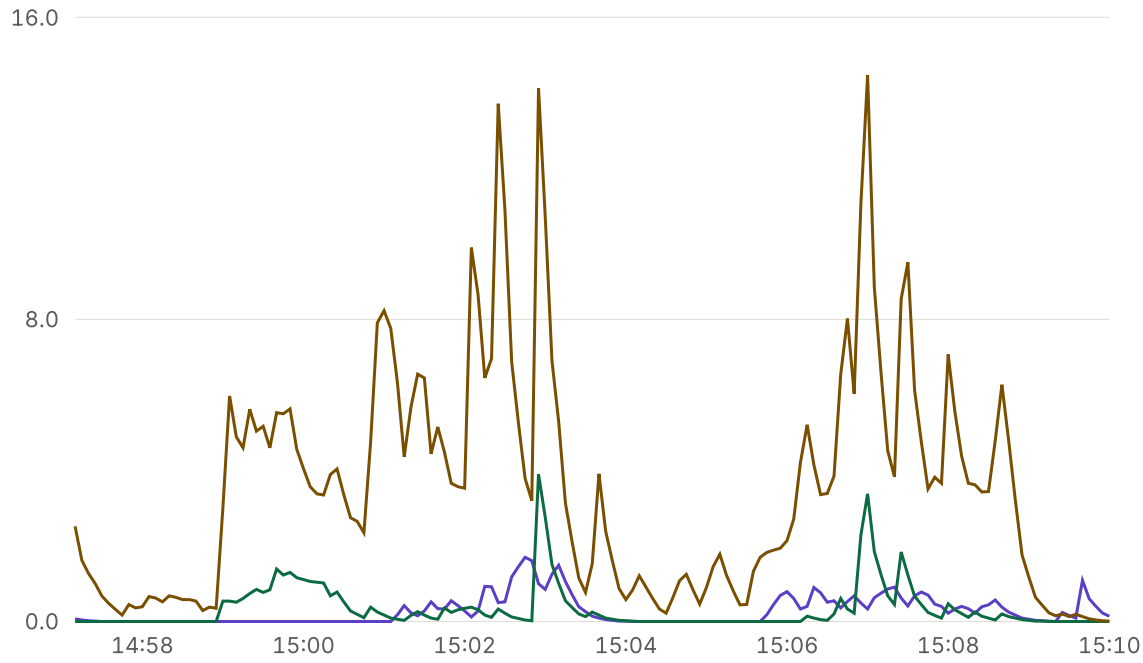
Series	p50	p75	p95	p99	Current
1. Received	0.0	0.0	0.0	0.0	0.0
2. Sent	0.0	0.0	0.0	0.0	0.0

Pressure stall, some tasks, 10 s average (%)

Share of time at least one task stalled waiting for CPU, memory or I/O, as a 10 second exponential moving average. Linux 4.20 and newer report it in /proc/pressure. RHEL 8 needs the psi=1 boot option.

What to look for

Values near zero are healthy. A value that stays above 10% shows which resource held work back. Memory pressure above zero means tasks waited while the kernel freed memory. I/O pressure usually lines up with high disk latency.



Pressure stall, some tasks, 10 s average (%): summary

Series	p50	p75	p95	p99	Current
1. CPU	0.1	0.4	1.0	1.5	0.1
2. Memory	0.1	0.4	1.3	3.0	0.0
3. I/O	3.4	5.2	9.0	13.9	0.0

Filesystem usage

Space used on each local filesystem at capture time.

What to look for

Above 85% on the data or WAL filesystem leaves little room for growth, temporary files and bursts of WAL. If the WAL filesystem fills up, PostgreSQL shuts down.

point in time, local disks

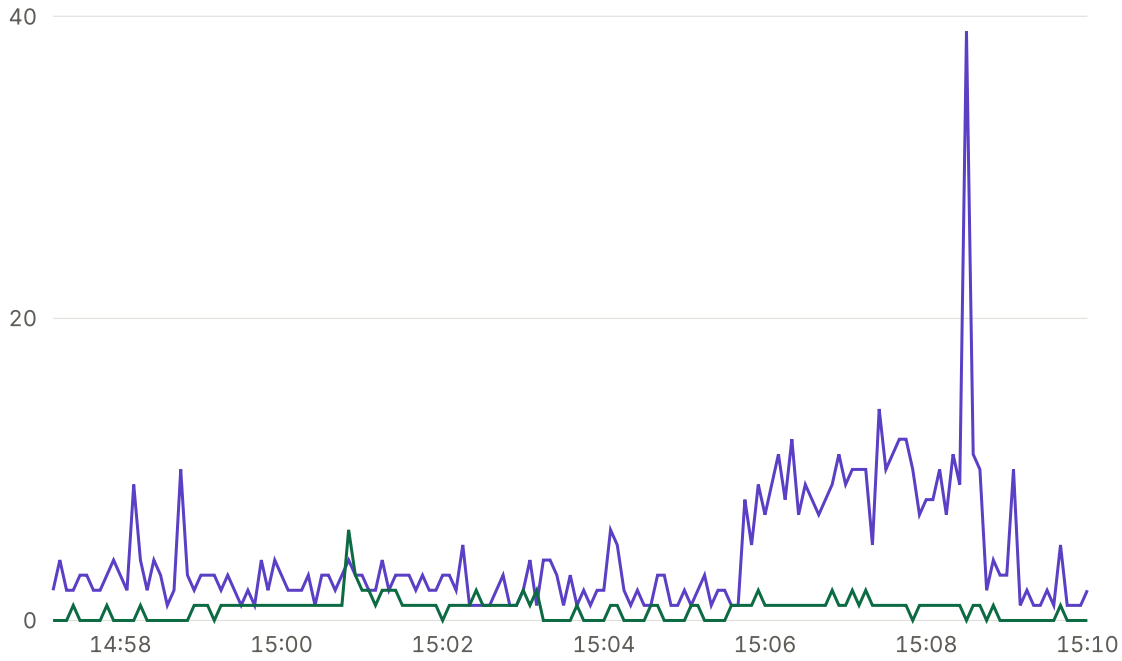
/artifacts (btrfs)	434.5 GiB of 463.7 GiB
/workload (btrfs)	434.5 GiB of 463.7 GiB
/etc/hostname (btrfs)	434.5 GiB of 463.7 GiB
/etc/hosts (btrfs)	434.5 GiB of 463.7 GiB
/etc/resolv.conf (btrfs)	434.5 GiB of 463.7 GiB
/var/lib/postgresql/data (btrfs)	434.5 GiB of 463.7 GiB

Processes running and blocked

Processes running on a CPU, and processes blocked, which on Linux mostly means waiting on disk.

What to look for

Running above the CPU count means tasks waited for a core. Blocked above zero for several samples in a row means tasks waited on storage. Compare with I/O wait in CPU utilization and with the disk latency charts.



Processes running and blocked: summary

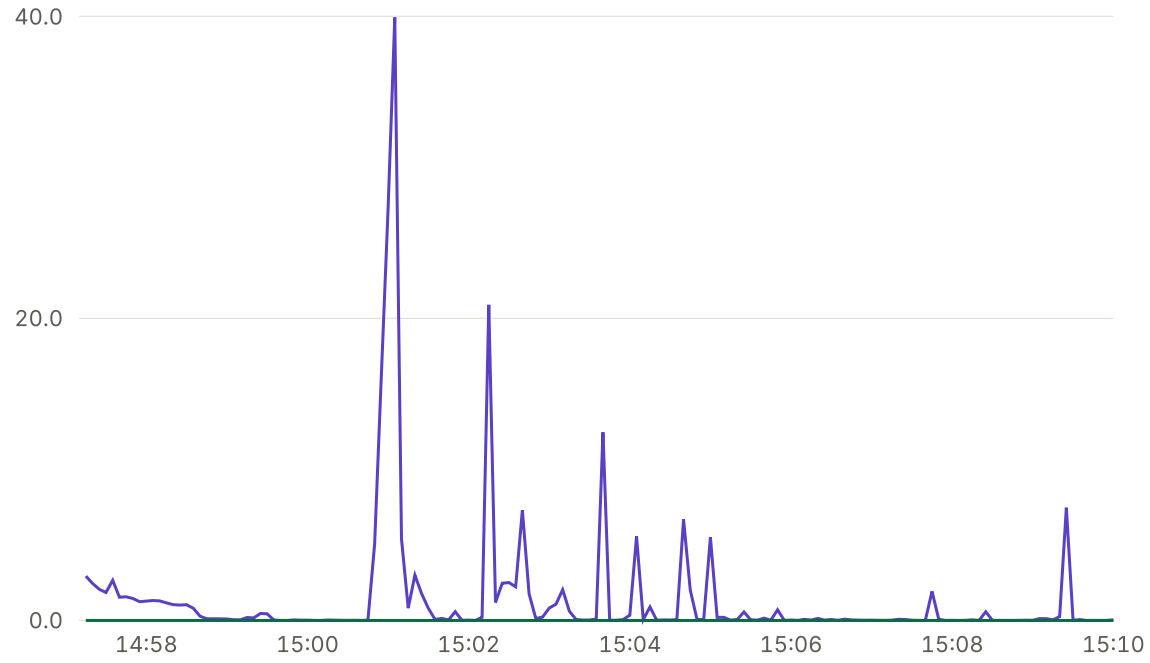
Series	p50	p75	p95	p99	Current
1. Running	3	5	11	13	2
2. Blocked on I/O	1	1	2	2	0

Swapped in and out (MiB/s)

Data the kernel moved from RAM to swap, and back from swap to RAM, per second.

What to look for

Swapped in is the line that hurts, because a process stops until each page comes back from disk. Any steady swap in during the session slows PostgreSQL. Swapped out alone means the kernel is making room, often for page cache.



Swapped in and out (MiB/s): summary

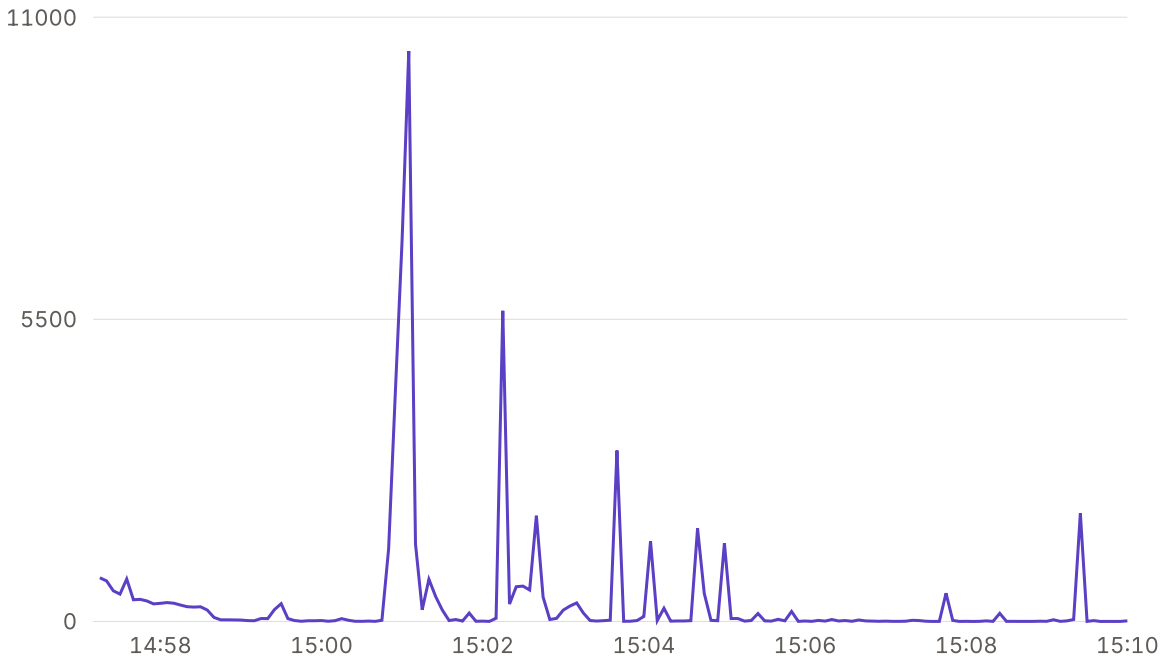
Series	p50	p75	p95	p99	Current
1. Swapped in	0.1	1.0	6.0	23.9	0.0
2. Swapped out	0.0	0.0	0.0	0.0	0.0

Major page faults per second

Page faults per second that needed a disk read, because the page was not in memory.

What to look for

A low, steady rate is normal when programs start or read memory mapped files. Spikes that line up with swap in or with a shrinking page cache mean the host was short of memory.



Major page faults per second: summary

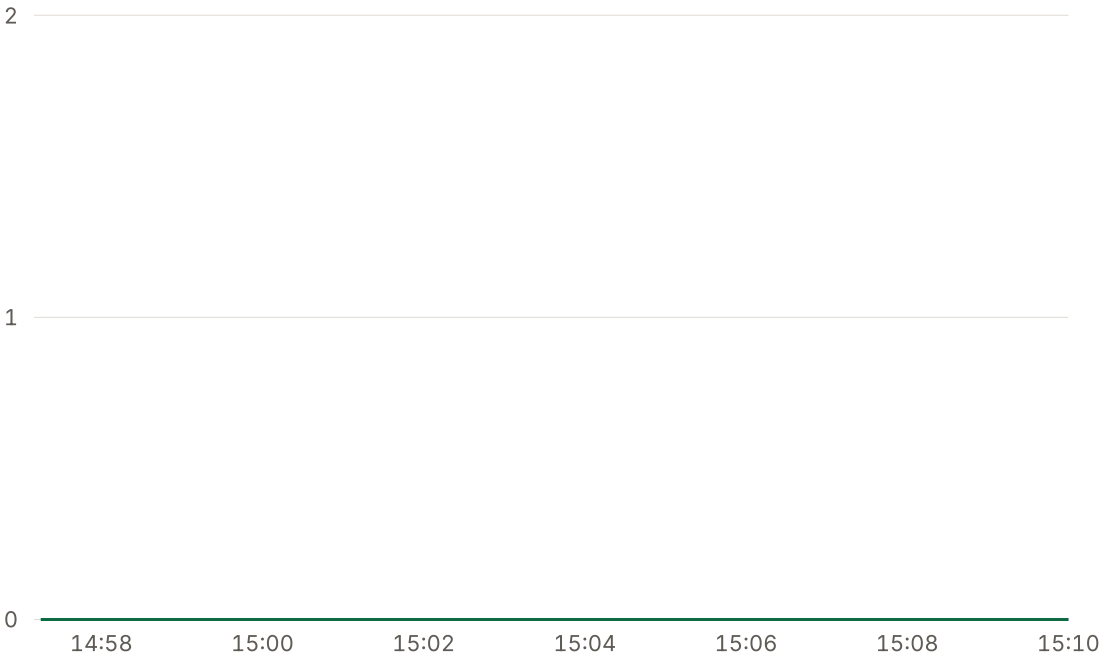
Series	p50	p75	p95	p99	Current
1. Major faults	19.4	265.6	1541.5	6251.7	11

Pages scanned for reclaim per second

Pages the kernel scanned per second to free memory. kswapd scans in the background. Direct reclaim happens when a process needs memory and has to free some itself before it can go on.

What to look for

Some kswapd activity is normal on a busy host. Direct reclaim above zero means processes, PostgreSQL backends included, stalled while they asked for memory. That causes latency spikes even when swap stays flat.



Pages scanned for reclaim per second: summary

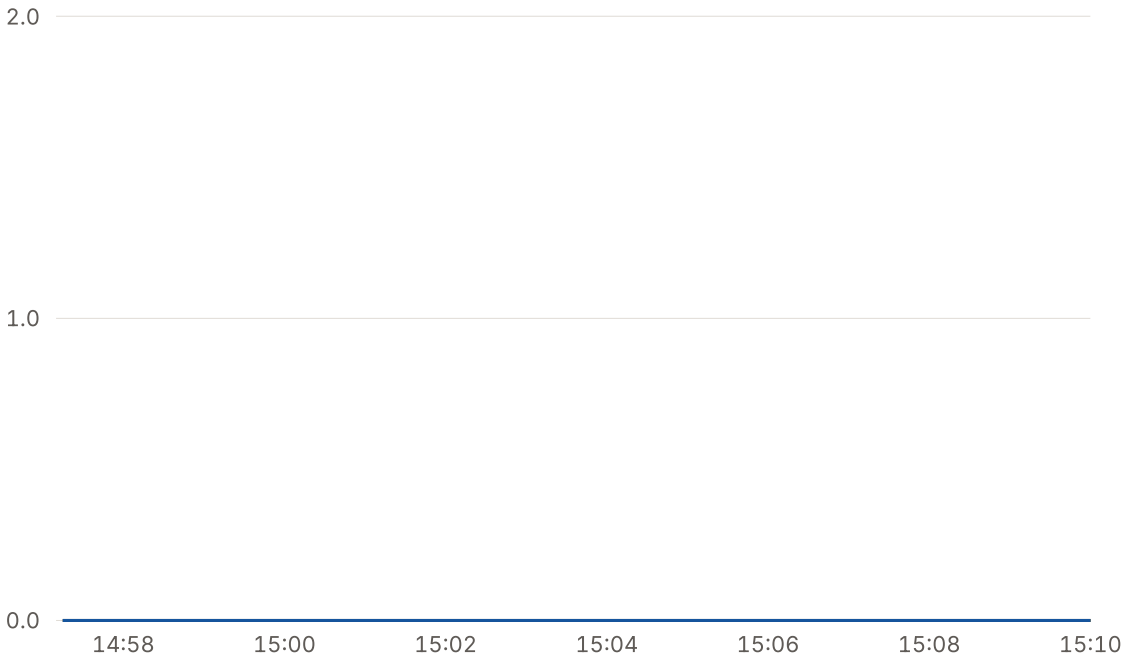
Series	p50	p75	p95	p99	Current
1. kswapd, in the background	0	0	0	0	0
2. Direct, by the allocating process	0	0	0	0	0

Packets dropped and errors per second, all interfaces

Packets dropped and interface errors per second, received and sent, over all interfaces.

What to look for

A low, steady rate of received drops can be frames the host does not handle, such as unknown protocols. Drops that rise with traffic can mean the interface buffers overflow under load. Errors point to a cable, driver or network card problem. A dropped TCP packet must be sent again, which slows clients and standbys.



Packets dropped and errors per second, all interfaces: summary

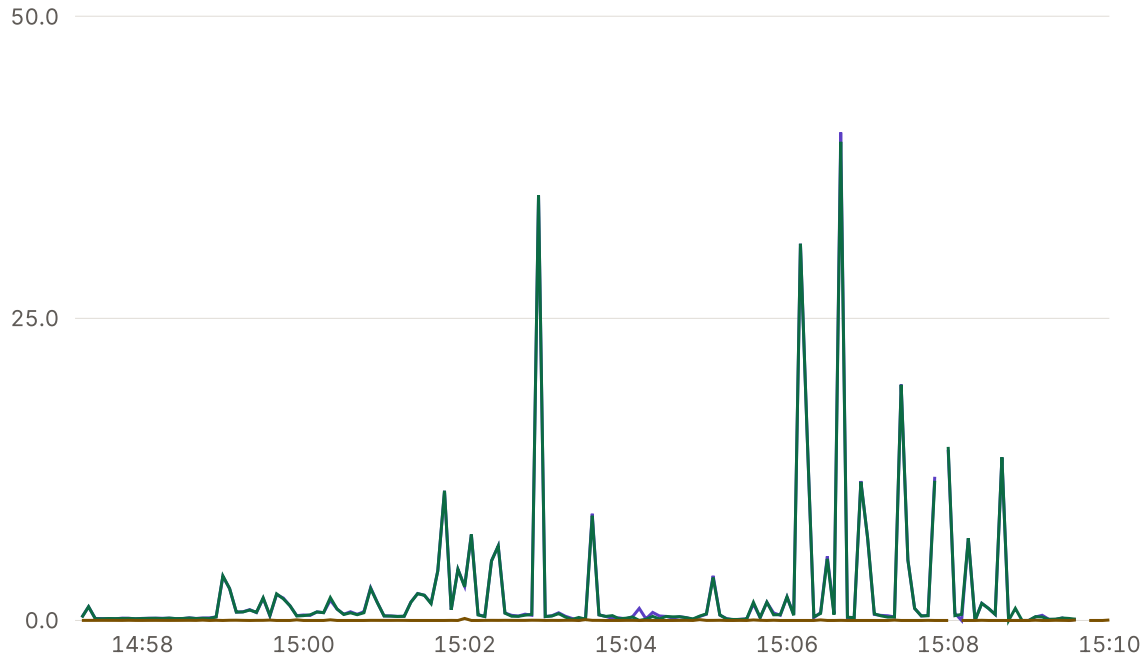
Series	p50	p75	p95	p99	Current
1. Received, dropped	0.0	0.0	0.0	0.0	0.0
2. Sent, dropped	0.0	0.0	0.0	0.0	0.0
3. Received, errors	0.0	0.0	0.0	0.0	0.0
4. Sent, errors	0.0	0.0	0.0	0.0	0.0

Average read latency, five busiest disks (ms)

Average time a read request took on each of the five busiest disks, in milliseconds.

What to look for

Local SSDs usually answer in under 1 ms, cloud volumes in 1 to 5 ms and spinning disks in 5 to 15 ms. Values in the tens of ms that last mean the disk was overloaded or throttled. The p95 in the table shows how slow the worst intervals were.



Average read latency, five busiest disks (ms): summary

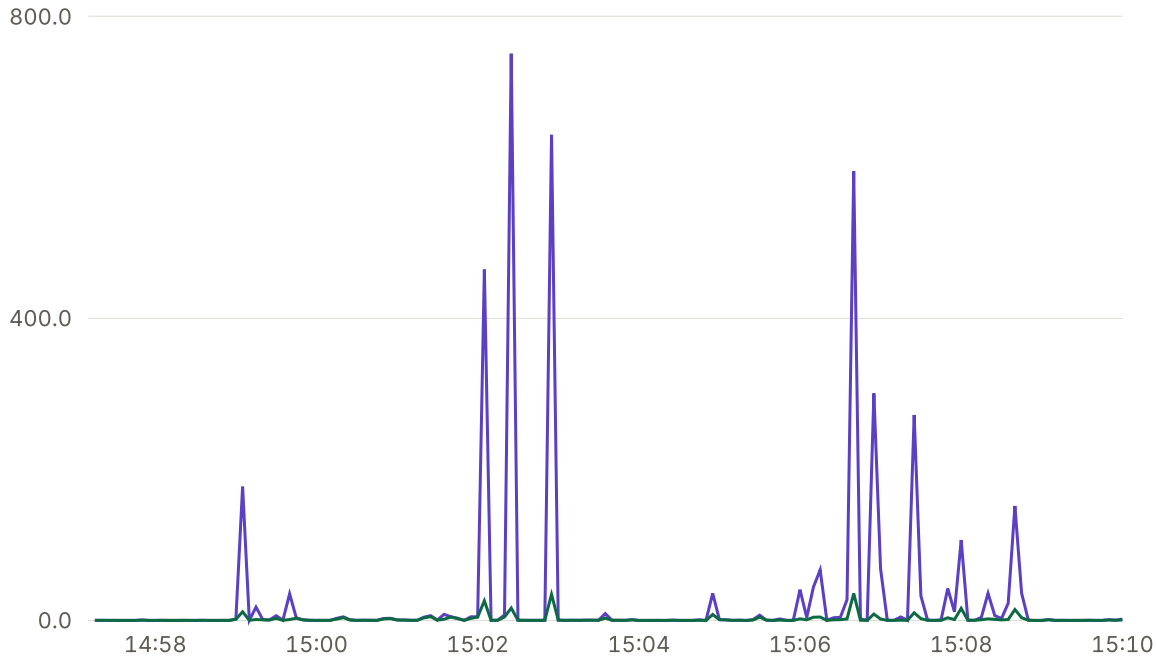
Series	p50	p75	p95	p99	Current
1. dm-0	0.4	1.4	11.8	33.3	Not collected
2. nvme0n1	0.4	1.4	11.5	33.3	Not collected
3. zram0	0.0	0.0	0.0	0.1	0.0

Average write latency, five busiest disks (ms)

Average time a write request took on each of the five busiest disks, in milliseconds.

What to look for

With synchronous_commit on, each commit waits until its WAL reaches the disk. High write latency on the WAL disk therefore slows every transaction. On SSD or cloud storage, writes above 10 ms make commits slow enough for users to notice. Spikes every few minutes often match checkpoints.



Average write latency, five busiest disks (ms): summary

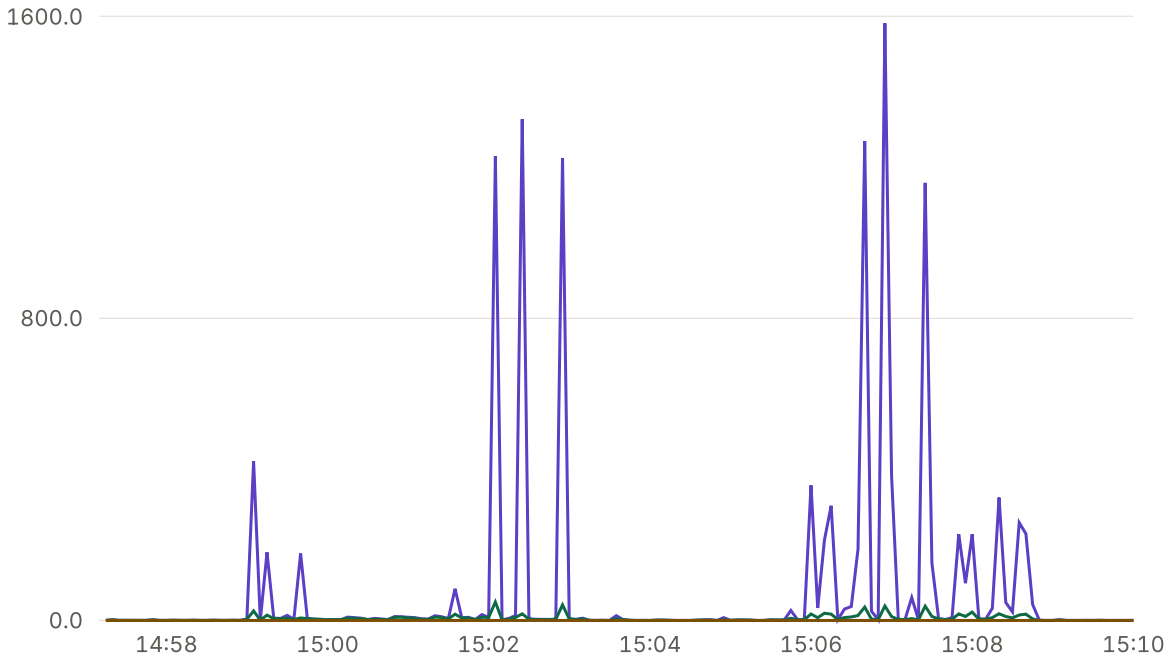
Series	p50	p75	p95	p99	Current
1. dm-0	0.4	3.9	122.2	617.5	1.9
2. nvme0n1	0.3	1.3	9.1	30.1	0.6
3. zram0	Not collected	Not collected	Not collected	Not collected	Not collected

Average requests in flight, five busiest disks

Average number of requests in flight on each of the five busiest disks.

What to look for

A queue that grows together with latency means the disk could not keep up. A deep queue with low latency is fine, because SSDs and cloud volumes work best with many requests at once.



Average requests in flight, five busiest disks: summary

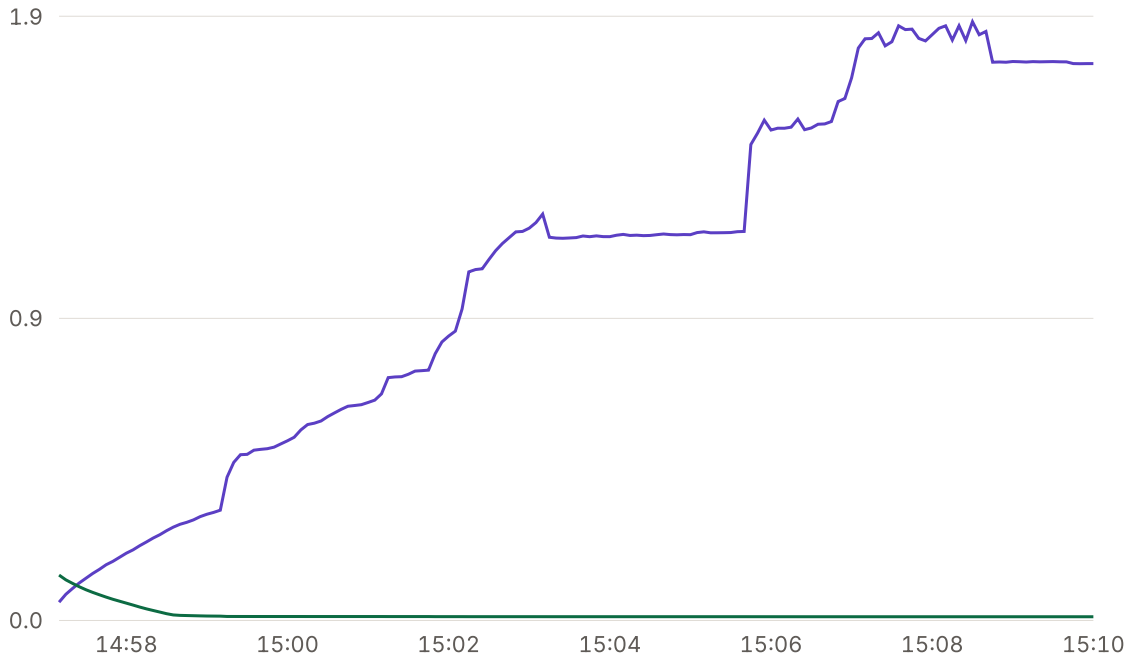
Series	p50	p75	p95	p99	Current
1. dm-0	2.6	9.8	366.3	1296.3	0.3
2. nvme0n1	2.3	6.0	18.3	39.7	0.1
3. zram0	0.0	0.0	0.0	0.0	0.0

Memory and swap charged to the cgroup (GiB)

Memory and swap charged to the cgroup of the probe. When the probe runs in the PostgreSQL container or systemd unit, this is the memory of that container.

What to look for

Near the limit, the kernel reclaims memory inside the cgroup. At the limit it reclaims harder, and it kills a process only if reclaim cannot free enough. Page cache counts against the limit too, so a cgroup can look full while most of it is cache.



Memory and swap charged to the cgroup (GiB): summary

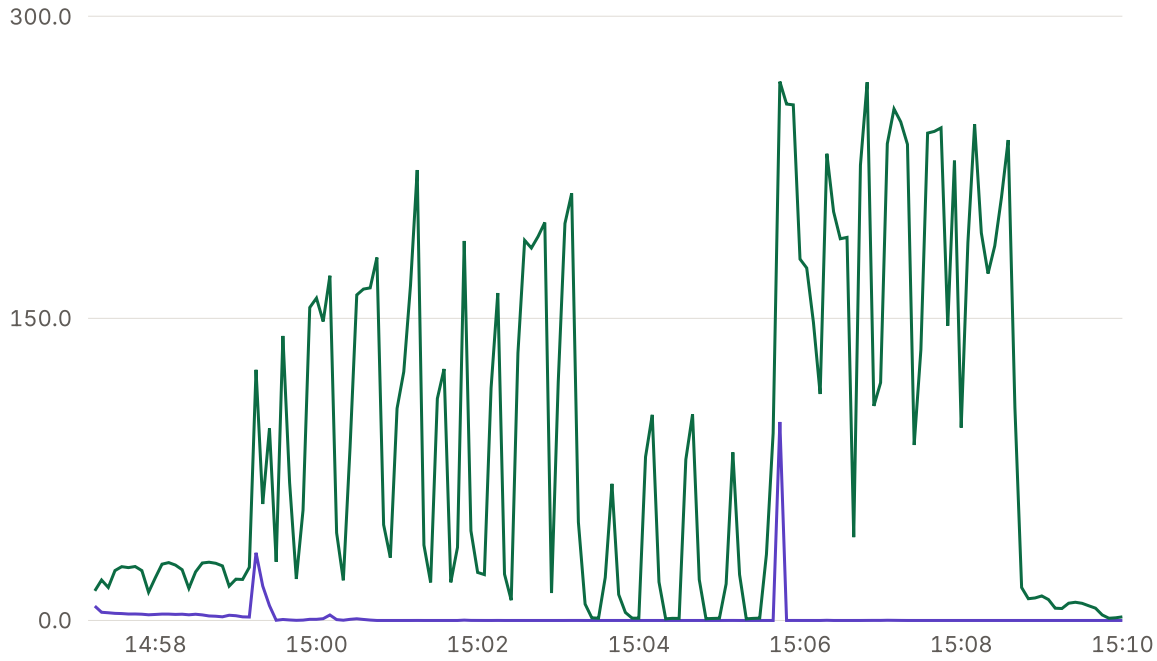
Series	p50	p75	p95	p99	Current
1. Memory	1.2	1.6	1.8	1.9	1.8
2. Swap	0.0	0.0	0.1	0.1	0.0

Cgroup I/O, all devices (MiB/s)

Bytes read and written per second by the cgroup of the probe, all devices.

What to look for

A flat top can be an I/O limit set on the container. If Disk throughput for the whole host has room while this line is flat, the limit is on the container, not on the disk. On cgroup v1, the kernel does not charge buffered writes to the container, so this line misses most writes.



Cgroup I/O, all devices (MiB/s): summary

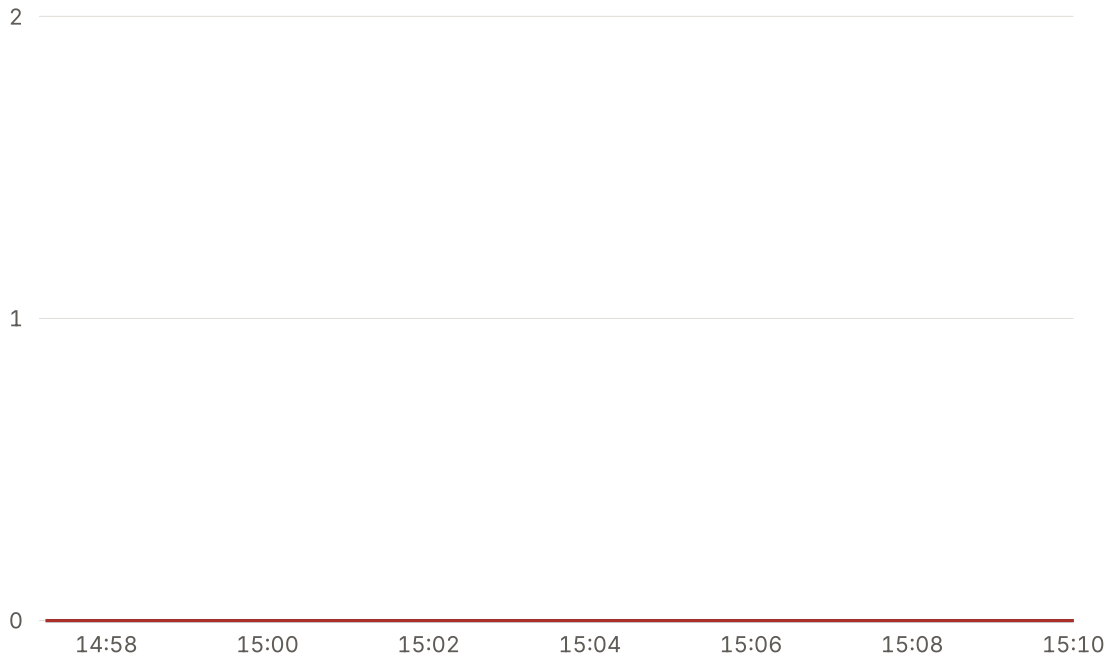
Series	p50	p75	p95	p99	Current
1. Read	0.0	0.2	3.5	24.8	0.0
2. Write	44.0	164.0	243.4	261.5	1.8

Cgroup limit events per interval

Change per interval of the limit counters of the cgroup of the probe, named controller.counter: cpu.nr_throttled, memory.high, memory.max, memory.oom_kill and pids.max. On cgroup v1, pg-probe records only CPU usage for now, so there the chart shows it, in microseconds of CPU time.

What to look for

cpu.nr_throttled above zero means the CPU limit of the cgroup paused its tasks. memory.high counts times the kernel throttled the cgroup and reclaimed memory at its soft limit, which never kills a process. memory.max counts times the cgroup reached its hard limit. The kernel kills a process only when reclaim fails there, and memory.oom_kill counts those kills. When a PostgreSQL backend is killed, the server ends every session and runs crash recovery. pids.max means the cgroup could not start a process, so new connections fail.



Cgroup limit events per interval: summary

Series	p50	p75	p95	p99	Current
1. cpu.nr_throttled	0	0	0	0	0
2. memory.high	0	0	0	0	0
3. memory.max	0	0	0	0	0
4. memory.oom_kill	0	0	0	0	0
5. pids.max	0	0	0	0	0

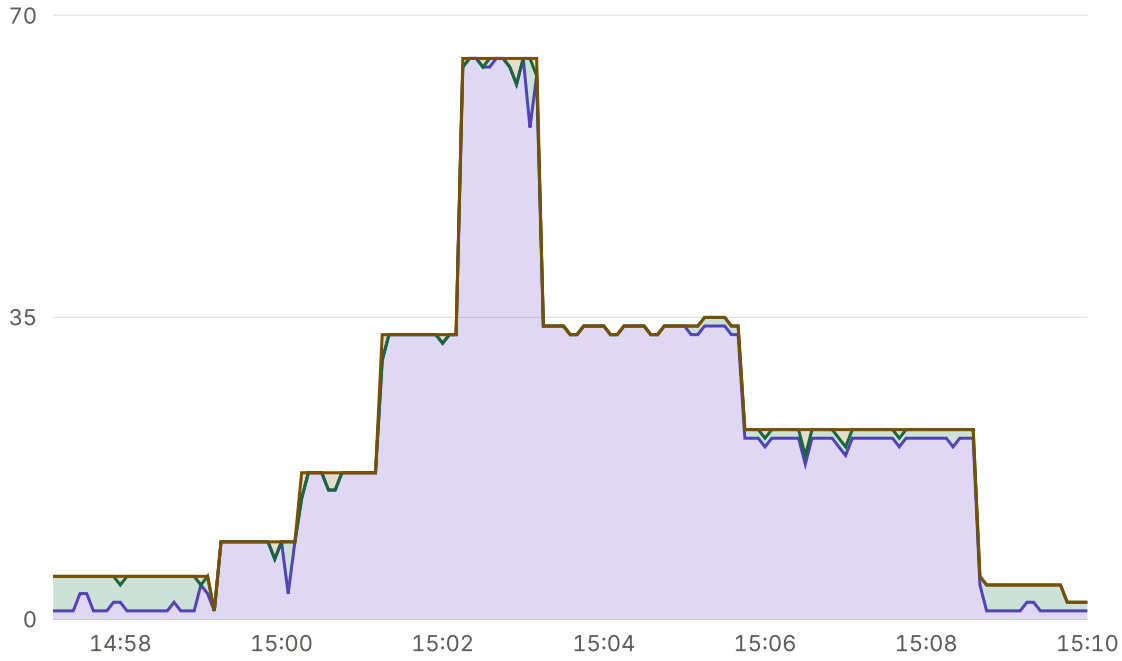
PostgreSQL

Client connections by state

Client connections at each sample, by state. Active connections run a query. Idle connections wait for the client. Idle in transaction connections have an open transaction but run nothing.

What to look for

Active connections well above the CPU count mean queries compete for cores. Idle in transaction connections that last hold locks and can keep vacuum from removing dead rows. Compare the total with max_connections on the Connections tile. Many idle connections suggest a connection pooler.



Client connections by state: summary

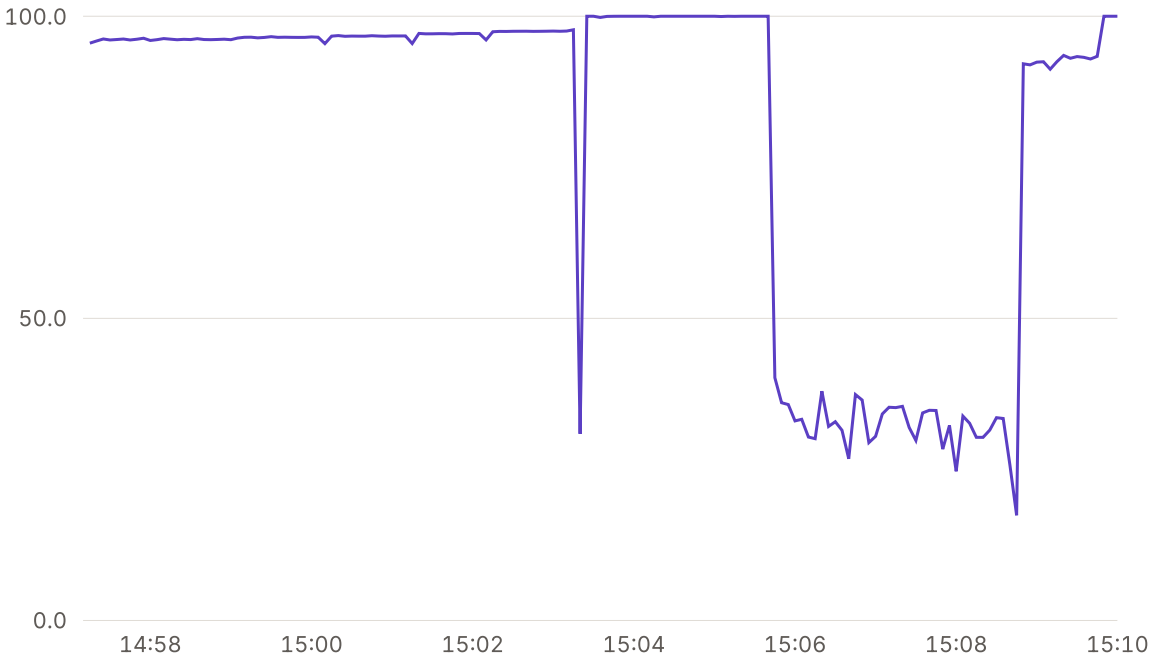
Series	p50	p75	p95	p99	Current
1. active	21	33	64	65	1
2. idle	1	1	4	5	1
3. idle in transaction	0	0	2	3	0

Cache hit ratio per interval (%)

Share of block reads PostgreSQL found in shared_buffers, per interval, over all databases. The Linux page cache can still serve a miss, so a miss is not always a disk read.

What to look for

OLTP workloads usually stay above 99%. Drops line up with scans of large tables, reports or a cold cache after a restart. A ratio that stays low while disk reads stay high means the working set does not fit in memory.



Cache hit ratio per interval (%): summary

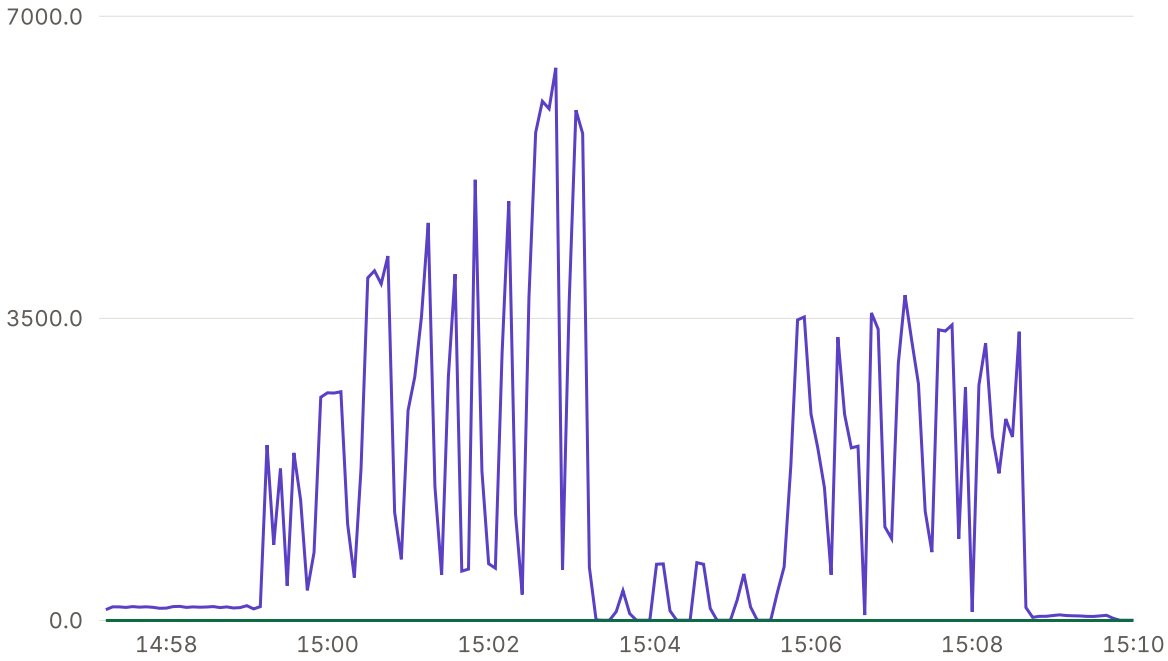
Series	p50	p75	p95	p99	Current
1. All databases	96.5	97.5	100.0	100.0	100.0

Transactions per second

Transactions committed and rolled back per second, over all databases.

What to look for

Rollbacks above a few percent of commits often point to application errors, deadlocks or timeouts. A sudden drop in commits while connections stay active can be a stall, such as lock waits or a slow disk. Compare the shape with the CPU and disk charts to see which resource the load uses most.



Transactions per second: summary

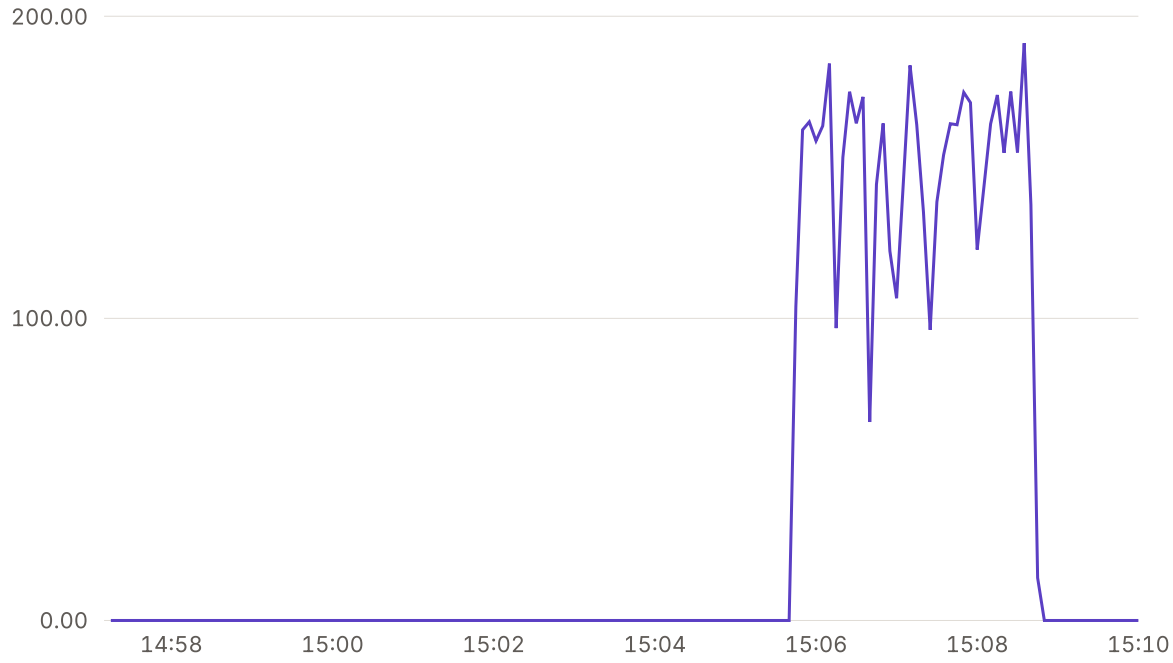
Series	p50	p75	p95	p99	Current
1. Commits	610.6	2622.7	4693.9	5969.0	2.6
2. Rollbacks	0.0	0.0	0.0	0.0	0.0

Temporary files written (MiB/s)

Data written to temporary files per second. PostgreSQL writes them when a sort, hash or similar step does not fit in work_mem.

What to look for

Spikes from a few large reports are normal. Steady temporary writes mean some queries need more work_mem, an index or a better plan. Set log_temp_files to find the queries that write them.



Temporary files written (MiB/s): summary

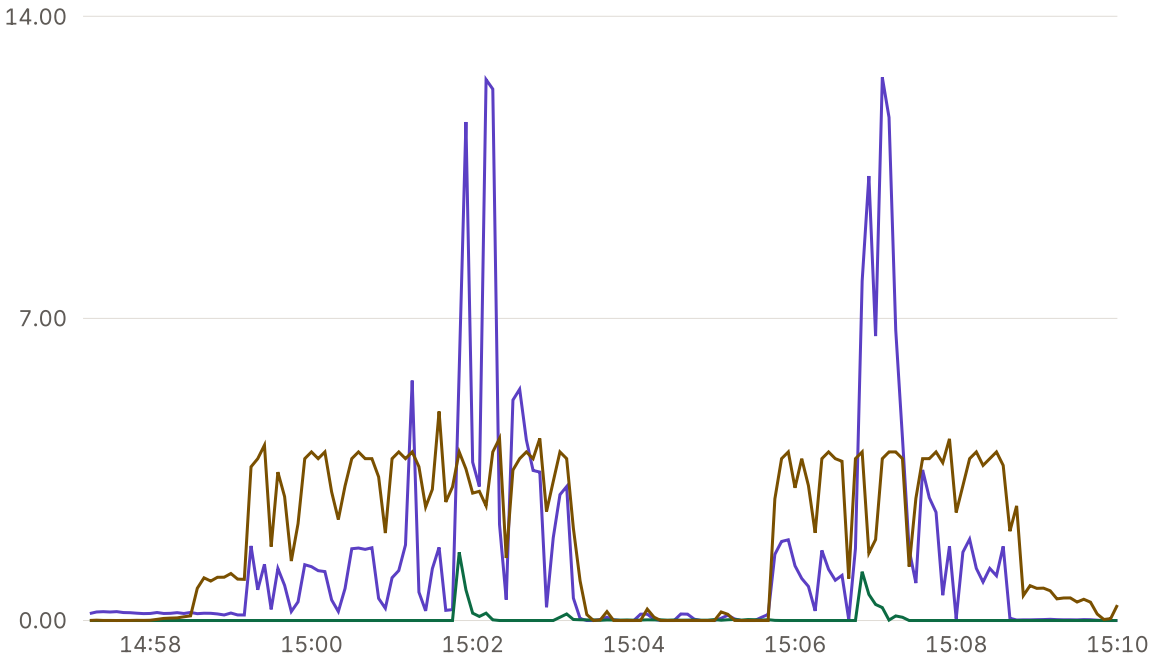
Series	p50	p75	p95	p99	Current
1. All databases	0.00	0.00	172.07	184.01	0.00

WAL and buffer writes (MiB/s)

WAL generated per second, and data pages written by the checkpointer, the background writer and the backends themselves.

What to look for

Most page writes should come from the checkpointer. Backend writes mean a query had to write a dirty page before it could reuse the buffer, and it waited for that write. Up to PostgreSQL 16, backend writes that persist outside bulk loads and VACUUM point to a small shared_buffers or a background writer that falls behind. Large checkpoint spikes can saturate the disk, so compare with disk latency.



WAL and buffer writes (MiB/s): summary

Series	p50	p75	p95	p99	Current
1. WAL generated	0.24	1.65	6.64	12.42	0.00
2. Checkpoint writes	0.00	0.01	0.16	0.91	0.00
3. Background writer	2.05	3.75	3.91	4.22	0.36
4. Backend writes	Not collected	Not collected	Not collected	Not collected	Not collected

Replication lag

last sample, from pg_stat_replication
No standby was connected in the last sample.

Database size

Size on disk of each database at capture time.

What to look for

Compare the total with the free space on the data filesystem. Compare with earlier captures to get the growth rate. Fast growth without new data can be bloat, so check Dead tuples.

point in time

postgres	568.4 MiB
template1	7.4 MiB
template0	7.2 MiB

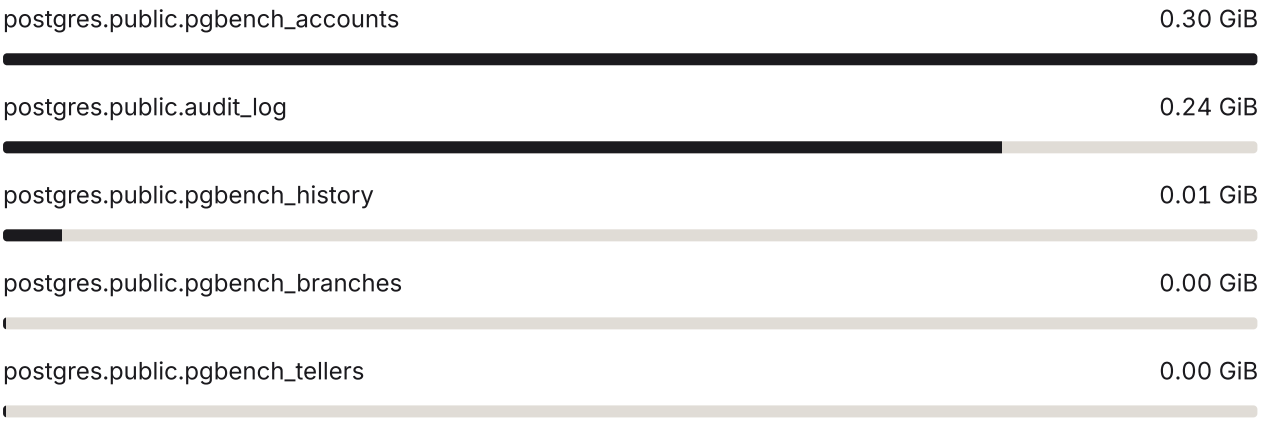
Largest tables

The six largest tables. Each bar is the estimated size of the table with its indexes and TOAST data, from the page counts in pg_class.

What to look for

The page counts update at VACUUM, ANALYZE and some DDL, so a table that grew since then shows smaller than it is. A table that also appears in Dead tuples needs vacuum tuning. A full scan of these tables costs the most I/O.

heap, indexes and TOAST, top 6



Dead tuples

The six tables with the most dead rows. A dead row is an old row version that an UPDATE or DELETE left behind and that vacuum has not removed yet.

> **What to look for**
>
> A high share of dead rows, or a last autovacuum long ago, means autovacuum falls behind. The table then grows, and scans read more pages. Long transactions and idle in transaction sessions keep vacuum from removing rows.

vacuum backlog, top 6

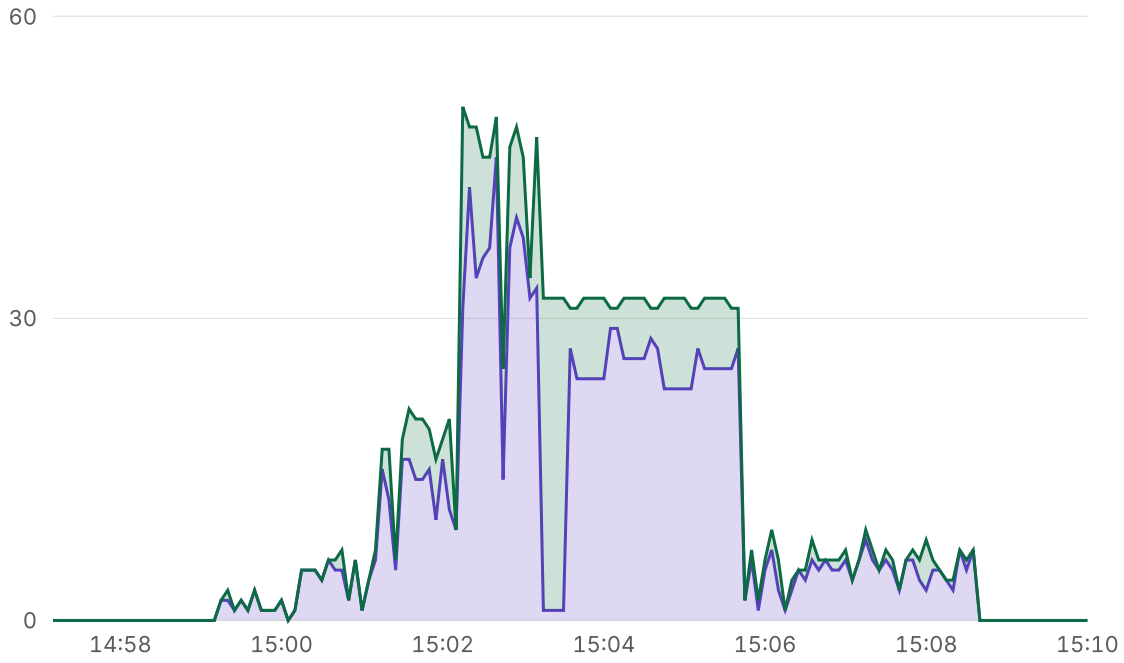
Table	Dead tuples	Live tuples	Dead (%)	Last autovacuum
`postgres.public.pgbench_accounts`	37,170	2,000,015	1.8	NULL

Lock requests waiting, by lock type

Lock requests waiting at each sample, by lock type.

What to look for

Short waits come and go on a busy server. A count that stays above zero means sessions block each other. Relation waits often come from DDL or LOCK TABLE. Transactionid waits mean rows locked by a long transaction.



Lock requests waiting, by lock type: summary

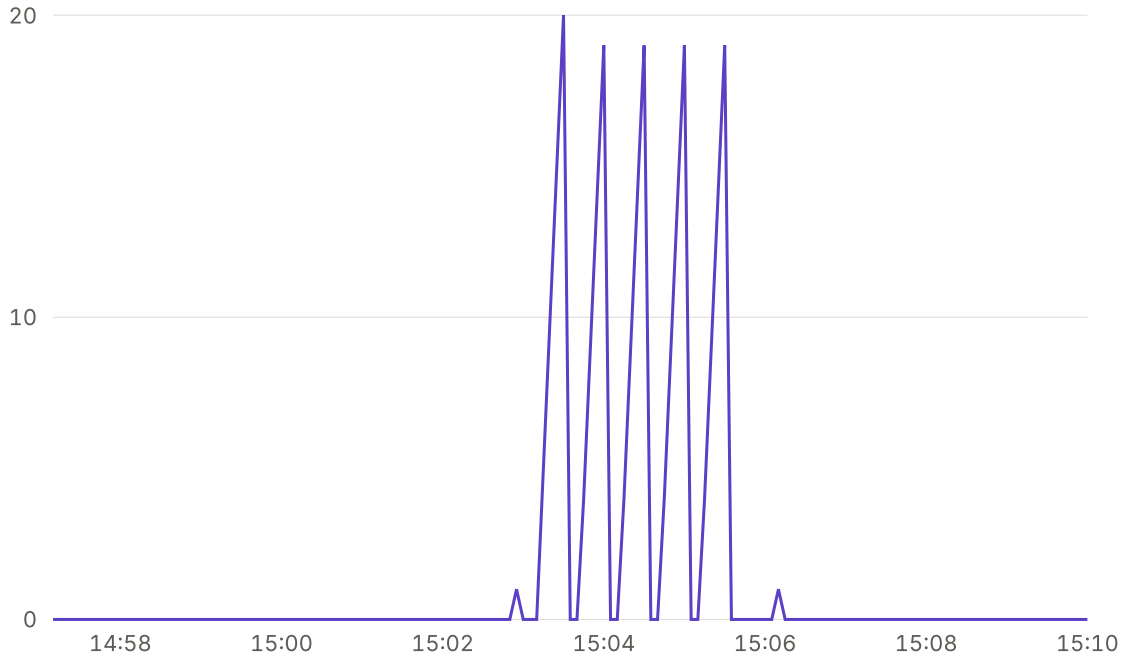
Series	p50	p75	p95	p99	Current
1. transactionid	5	16	33.3	41.4	0
2. tuple	1	5	10.3	31	0

Longest lock wait (s)

The longest time any request had waited for a lock at each sample, in seconds.

What to look for

Waits of more than a few seconds block users. One long wait can build a queue, because later requests wait behind it. pg_blocking_pids() finds the session that holds the lock.



Longest lock wait (s): summary

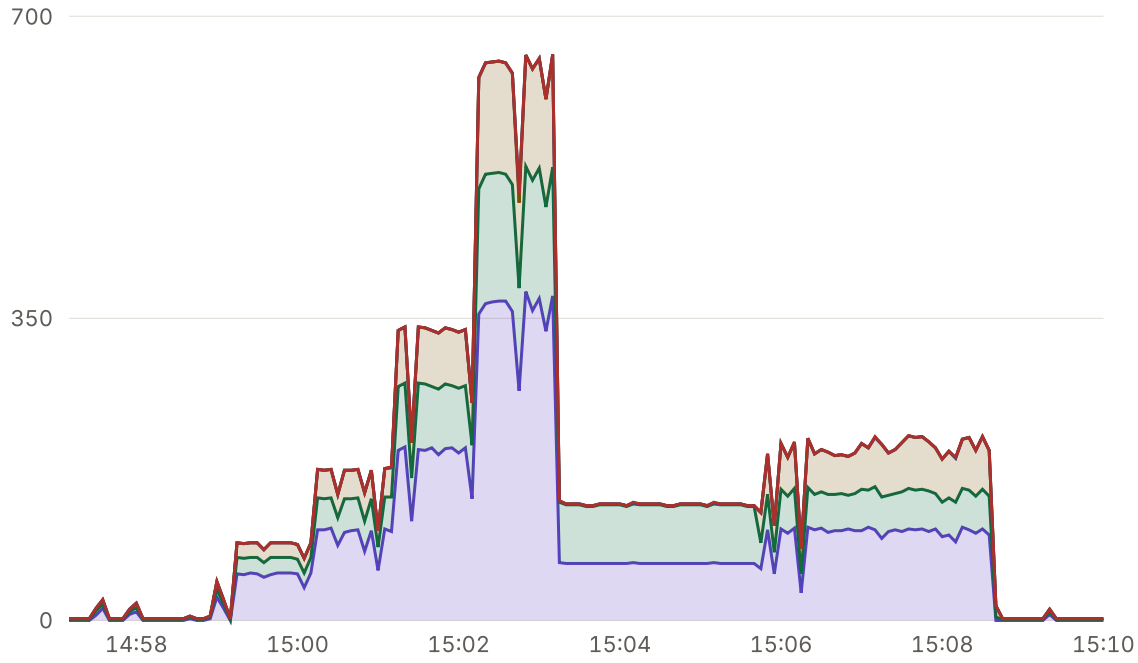
Series	p50	p75	p95	p99	Current
1. Longest wait	0	0	14	19	0

Locks held, by mode

Locks held at each sample, by lock mode.

What to look for

AccessShareLock comes from plain reads and is normal. AccessExclusiveLock blocks every read and write on its table. DDL, VACUUM FULL, TRUNCATE and some ALTER TABLE commands take it. A count of any mode that stays high can mean long transactions.



Locks held, by mode: summary

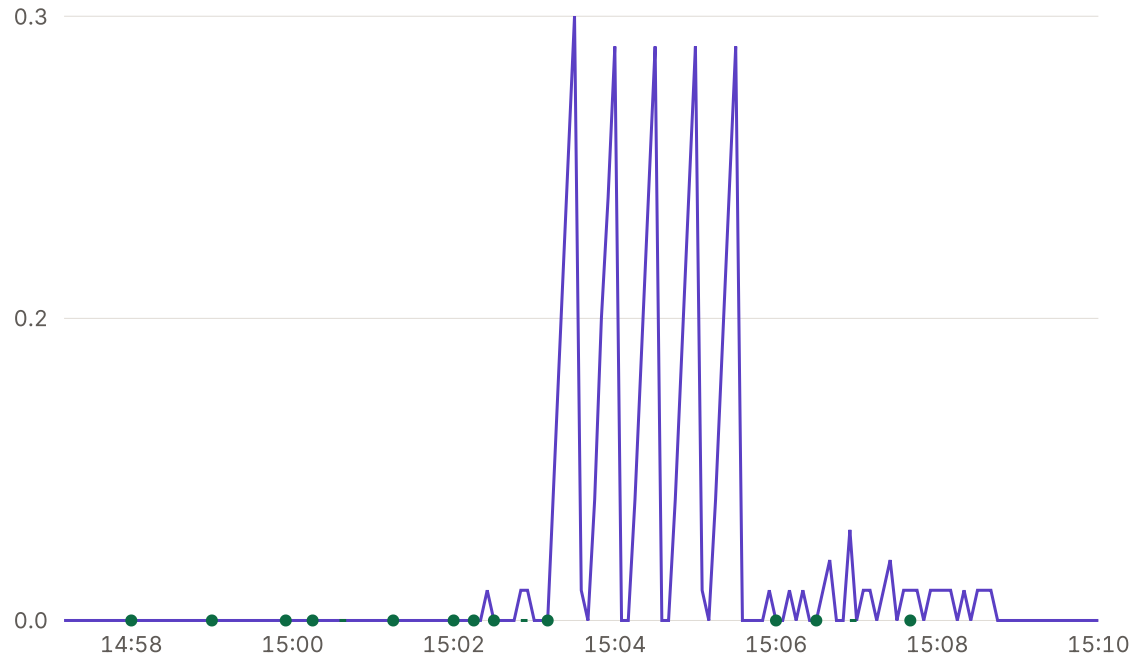
Series	p50	p75	p95	p99	Current
1. RowExclusiveLock	66	105	358.3	374.4	0
2. ExclusiveLock	42	68	145.6	150.5	1
3. AccessShareLock	17	53	129	129	1
4. ShareLock	0	0	0	1.5	0
5. ShareUpdateExclusiveLock	0	0	0.3	1	0

Longest time in state since state_change (min)

For active and idle in transaction sessions, the longest time any client session had stayed in that state since its last state change, in minutes.

What to look for

A long active time is a slow query. A long idle in transaction time holds locks and can block vacuum. From PostgreSQL 9.6, idle_in_transaction_session_timeout can end those sessions.



Longest time in state since state_change (min): summary

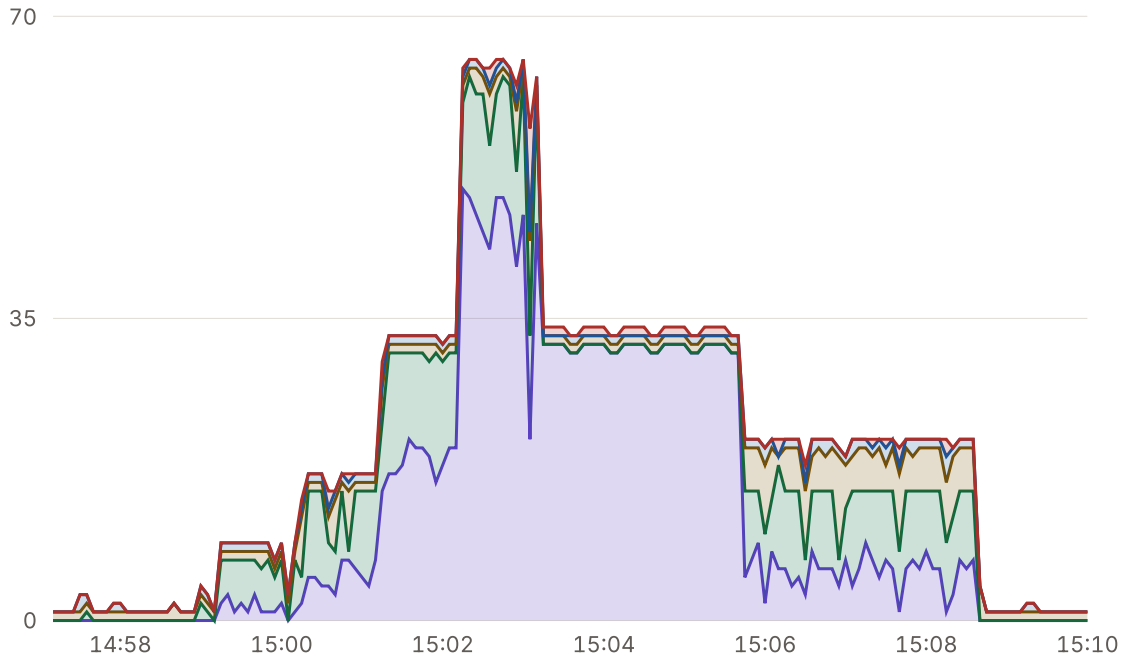
Series	p50	p75	p95	p99	Current
1. active	0.0	0.0	0.2	0.3	0.0
2. idle in transaction	0.0	0.0	0.0	0.0	Not collected

Active backends, by wait event type

Active backends at each sample, by the type of wait event they reported. No wait event means PostgreSQL reported none. The backend was running on a CPU, waiting for the operating system to schedule it, or in a wait PostgreSQL does not report, such as disk I/O on 9.6, which has no IO wait type.

What to look for

Lock waits mean sessions blocked each other. IO waits are time spent waiting for reads, writes or flushes, so compare them with the disk latency charts. LWLock waits point to contention inside PostgreSQL, for example on WAL or on the buffer mapping. Many backends with no wait event on a busy host can mean they queue for CPU.



Active backends, by wait event type: summary

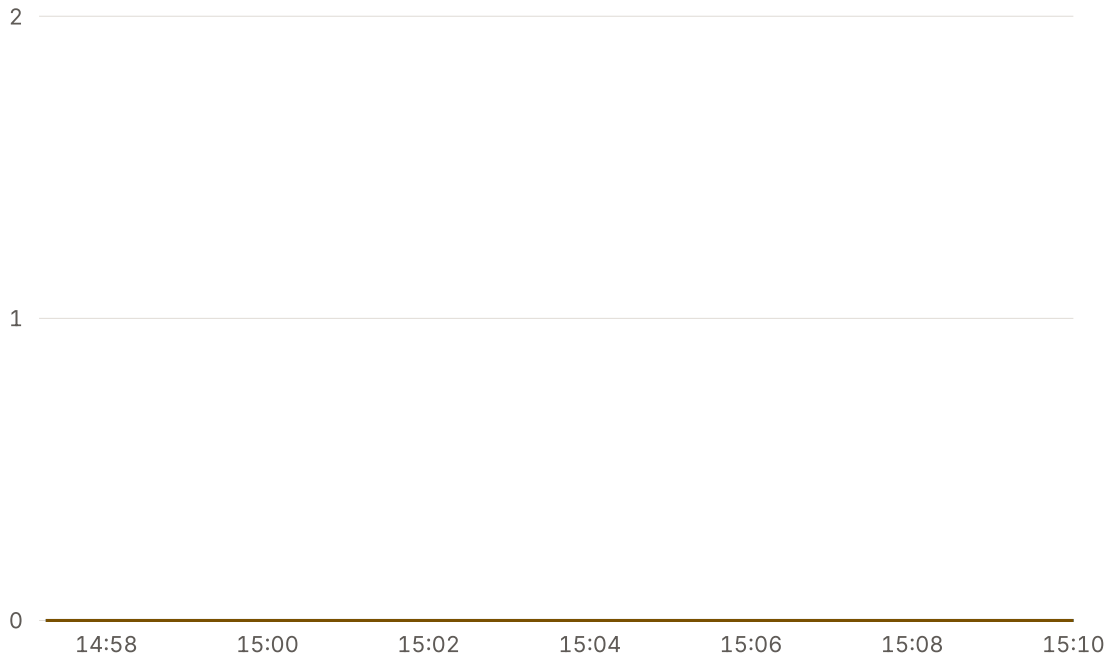
Series	p50	p75	p95	p99	Current
1. Lock	6	31	45.3	49	0
2. LWLock	4	9	14	15.5	0
3. Not waiting	1	4	7	9.9	1
4. IO	1	1	1.3	2	0
5. Other	0	0	2	2	0

Deadlocks, recovery conflicts and checksum failures per interval

Deadlocks, recovery conflicts and checksum failures per interval, over all databases.

What to look for

Each deadlock aborts a transaction. The fix is in the application, usually by taking locks in the same order everywhere. Recovery conflicts happen on standbys. From PostgreSQL 12, with data checksums on, any checksum failure means data on disk is corrupt. Check the server log and the storage the same day.



Deadlocks, recovery conflicts and checksum failures per interval: summary

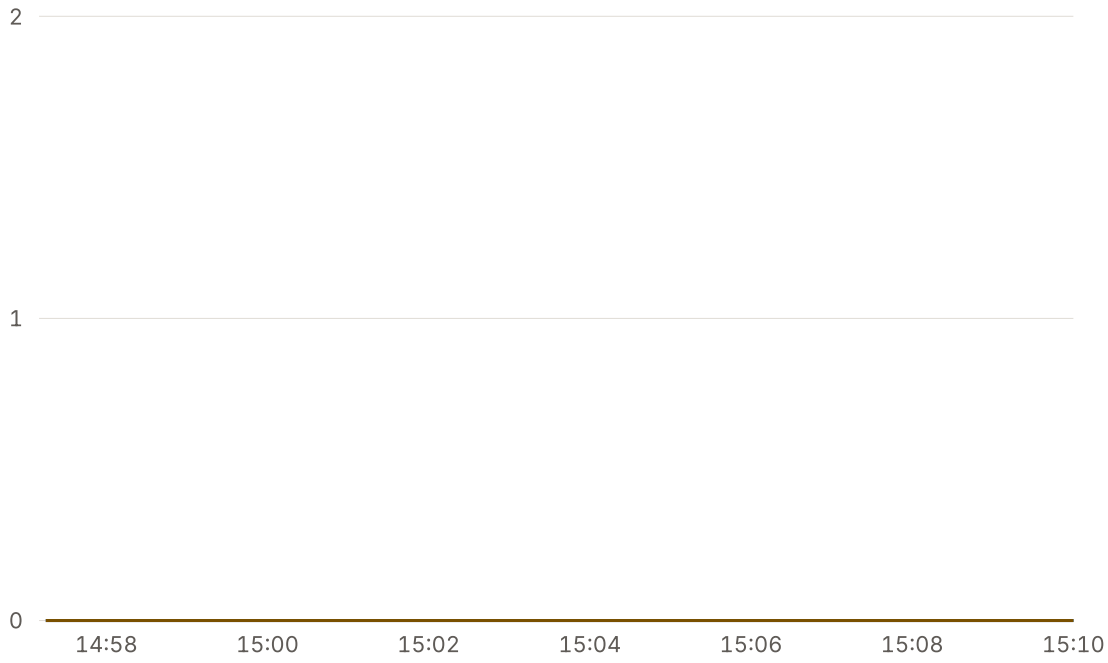
Series	p50	p75	p95	p99	Current
1. Deadlocks	0	0	0	0	0
2. Recovery conflicts	0	0	0	0	0
3. Checksum failures	0	0	0	0	0

Sessions ended abnormally per interval

Sessions per interval that ended without a clean disconnect. Abandoned means the client went away. Fatal means an error ended the session, timeouts such as idle_in_transaction_session_timeout included. Killed means a SIGTERM ended it, from pg_terminate_backend or a server shutdown.

What to look for

Abandoned sessions point to network problems, client crashes or a pooler that drops connections. Fatal sessions come from errors and timeouts, and the server log has the cause of each. Killed sessions match pg_terminate_backend calls by an operator or a tool, or a server shutdown.



Sessions ended abnormally per interval: summary

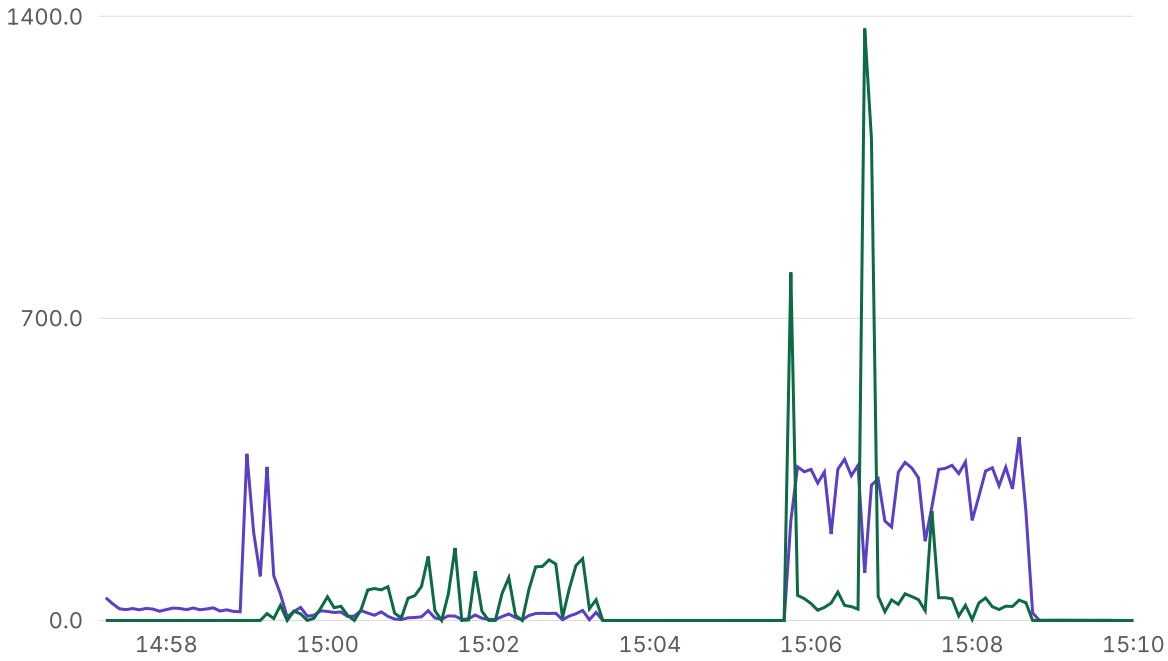
Series	p50	p75	p95	p99	Current
1. Abandoned	0	0	0	0	0
2. Fatal	0	0	0	0	0
3. Killed	0	0	0	0	0

Time reading and writing blocks (ms per second)

Time backends spent reading and writing data blocks, in milliseconds per second. It needs track_io_timing on.

What to look for

A value of 1000 means one backend, on average, spent all its time on I/O. Read time that rises while Cache hit ratio falls means the working set does not fit in memory. Compare with disk latency to tell a slow disk from too many reads.



Time reading and writing blocks (ms per second): summary

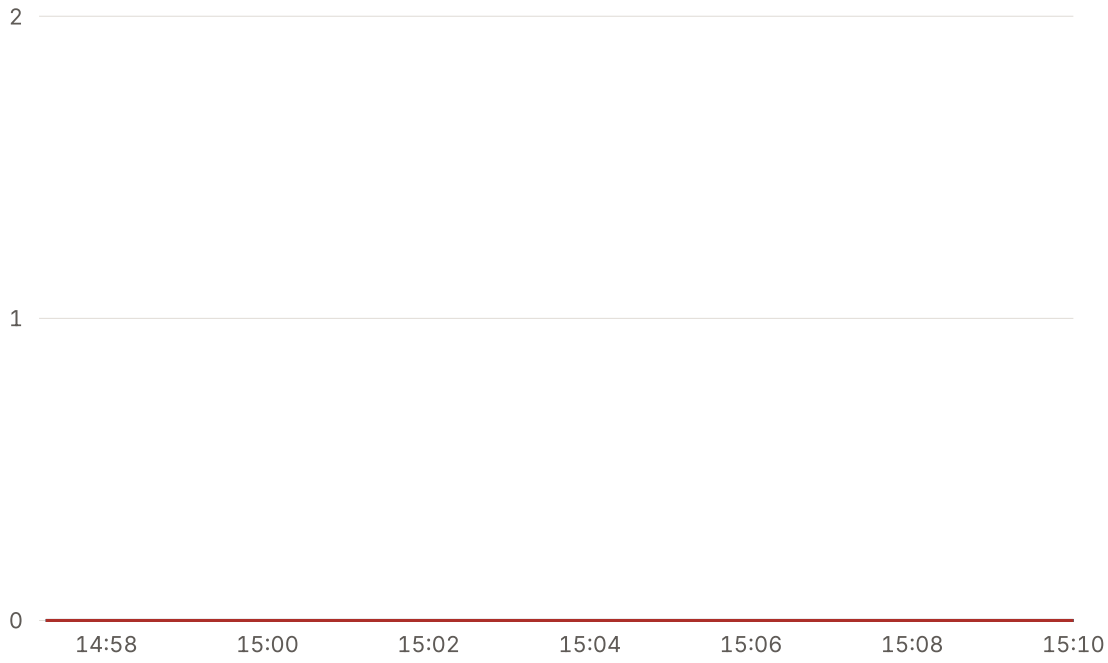
Series	p50	p75	p95	p99	Current
1. Read	16.7	108.6	355.9	379.5	0.0
2. Write	1.7	47.4	134.1	951.8	0.0

Queries canceled by recovery conflicts per interval

Queries on a standby that recovery canceled, per interval, by cause. A snapshot conflict happens when vacuum on the primary removed rows a standby query still needed.

What to look for

Any value above zero means standby users saw errors. Snapshot conflicts are the most common. hot_standby_feedback or a longer max_standby_streaming_delay reduces them, at the cost of more bloat on the primary or more lag on the standby.



Queries canceled by recovery conflicts per interval: summary

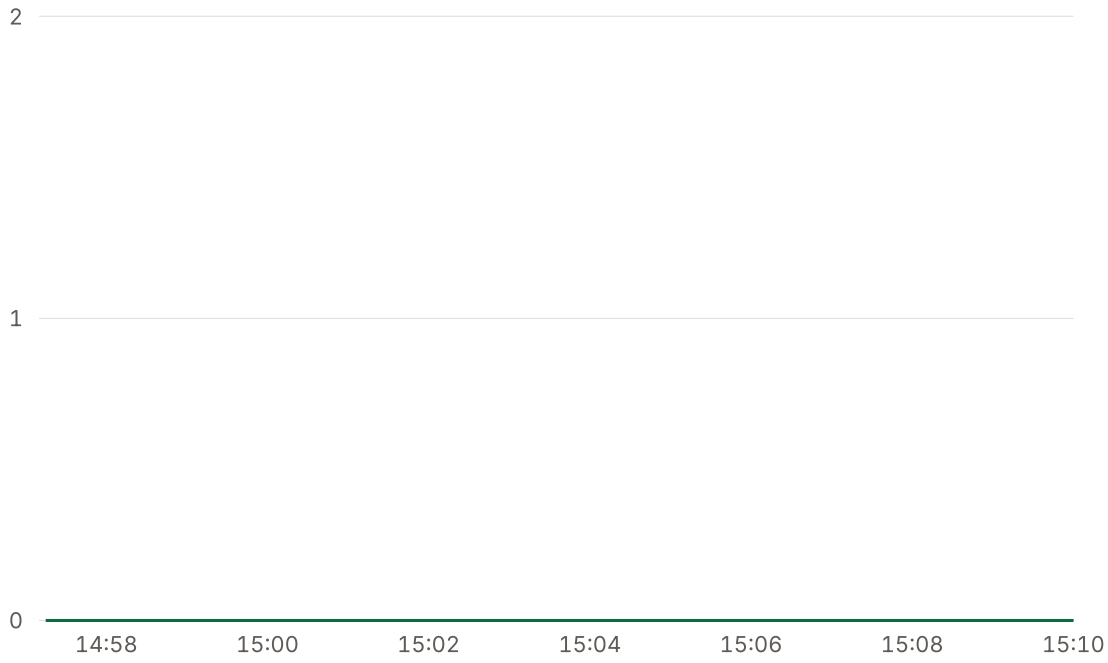
Series	p50	p75	p95	p99	Current
1. Snapshot	0	0	0	0	0
2. Lock	0	0	0	0	0
3. Buffer pin	0	0	0	0	0
4. Deadlock	0	0	0	0	0
5. Tablespace	0	0	0	0	0

WAL files archived per interval

WAL files archived, and archive failures, per interval, from pg_stat_archiver.

What to look for

Any failure means the archive falls behind, and point-in-time recovery cannot go past the last archived file. PostgreSQL keeps every WAL file it could not archive, so pg_wal (pg_xlog before PostgreSQL 10) grows until the archive command works again and can fill the disk. Archived files should keep pace with the WAL generated.



WAL files archived per interval: summary

Series	p50	p75	p95	p99	Current
1. Archived	0	0	0	0	0
2. Failed	0	0	0	0	0

WAL retained by replication slots (MiB)

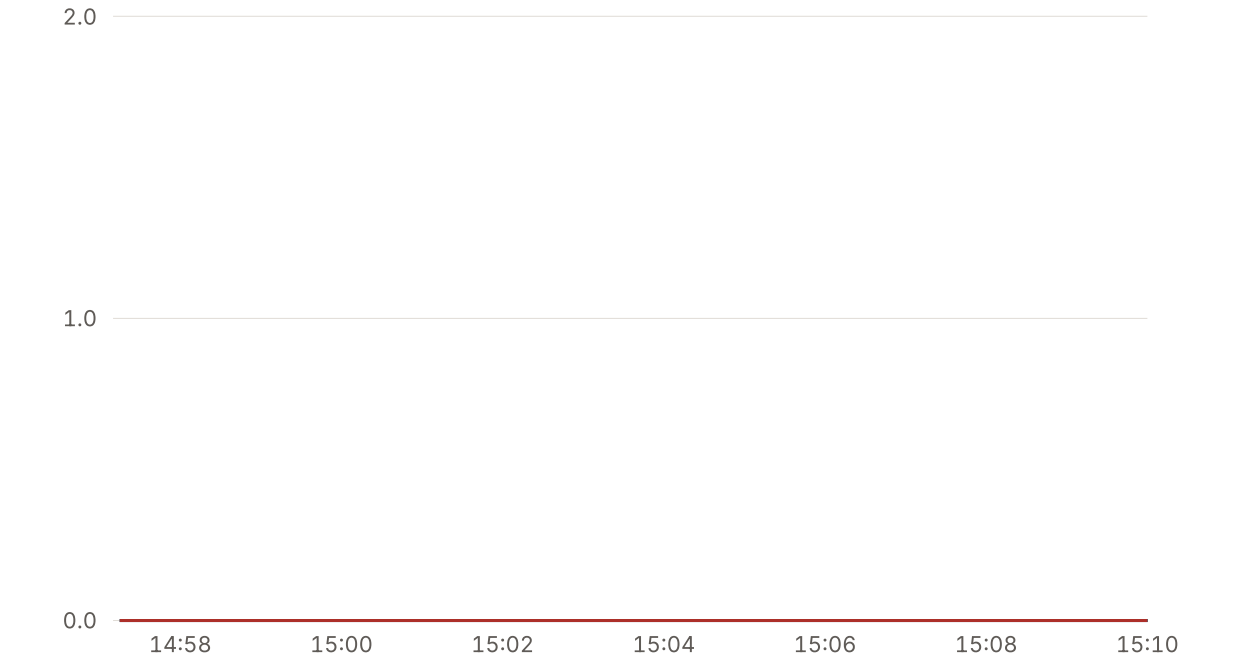
Not recorded in this artifact.

SLRU blocks read from disk per second

Blocks read from disk per second by the SLRU caches. These small caches hold transaction status, subtransactions, multixacts, commit timestamps, LISTEN/NOTIFY and serializable transactions.

What to look for

Values near zero are normal. Steady subtransaction reads come from heavy use of savepoints or of exception blocks in PL/pgSQL, and they can slow the whole server. Multixact reads come from many sessions locking the same rows, for example through SELECT FOR SHARE or foreign keys.



SLRU blocks read from disk per second: summary

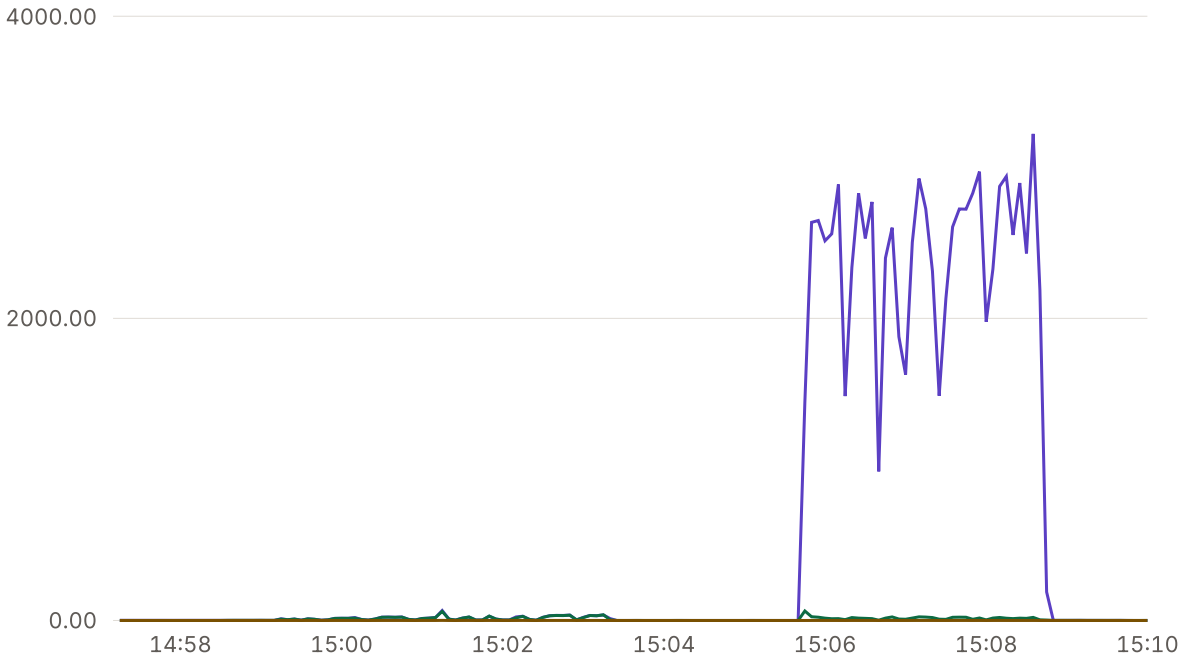
Series	p50	p75	p95	p99	Current
1. multixact_offset	0.0	0.0	0.0	0.0	0.0
2. transaction	0.0	0.0	0.0	0.0	0.0
3. commit_timestamp	0.0	0.0	0.0	0.0	0.0
4. multixact_member	0.0	0.0	0.0	0.0	0.0
5. Other	0.0	0.0	0.0	0.0	0.0

I/O by operation, all backends (MiB/s)

Data read, written and extended per second by all backends, from pg_stat_io. Extends are new blocks added to the end of a table or index file.

What to look for

Large reads point to scans of tables that do not fit in memory. Writes should follow the checkpoints and the background writer, so compare with WAL and checkpoint writes. Steady extends are normal for tables that grow.



I/O by operation, all backends (MiB/s): summary

Series	p50	p75	p95	p99	Current
1. Reads	3.80	36.28	2828.48	2954.54	0.00
2. Writes	3.15	15.38	31.02	48.20	0.36
3. Extends	0.02	0.13	0.26	0.35	0.00

Queries

Query source

stated, never guessed

pg_stat_statements is installed in postgres. The artifact does not read it yet. Some databases were not visited: unsupported (database_not_connectable).

Query latency

blocked

percentiles across the window

pg_stat_statements stores totals and means, never a distribution. Percentiles need the server log, which the probe does not read.

Query volume

no collector

calls per second

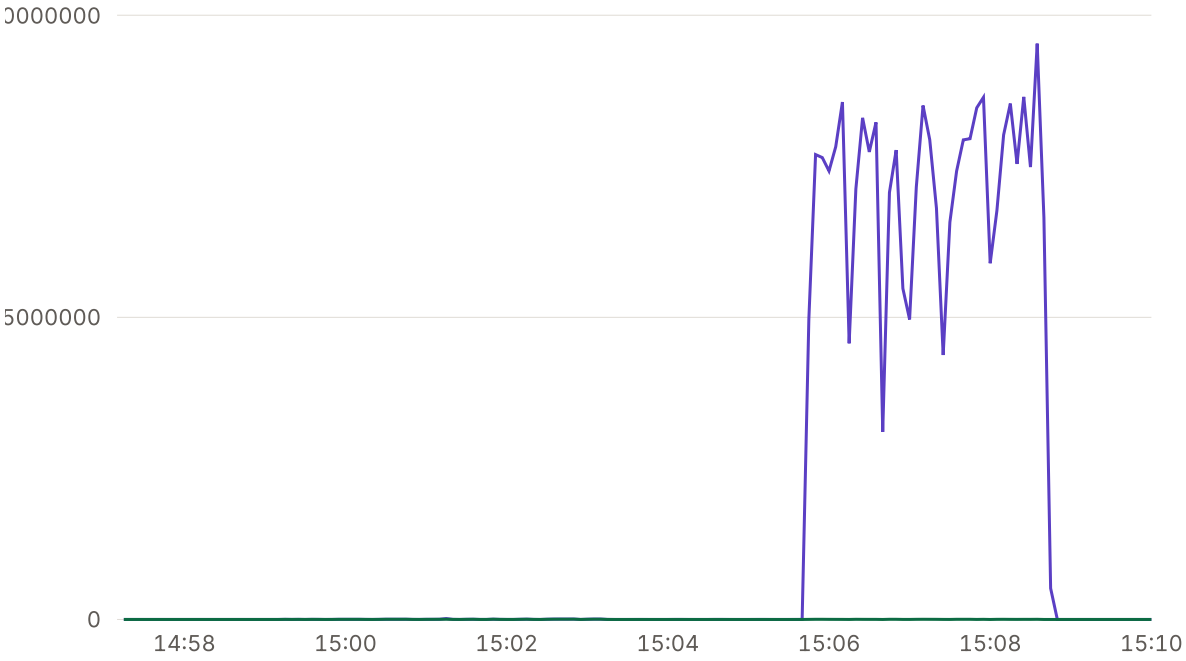
Needs a new postgres.stats.statements collector, after the decision on whether query text may enter the artifact.

Rows per second

Rows per second over all databases. Read is rows returned by scans. Written is rows inserted, updated and deleted.

What to look for

Read counts every row a scan returned, before filters, so a high read rate with few transactions points to sequential scans of large tables. A written rate that rises with WAL generated is normal. Compare the shape with Transactions per second.



Rows per second: summary

Series	p50	p75	p95	p99	Current
1. Read	3205	31757	41266480	43224233	2
2. Written	2070	10480	18657	23867	0

Time split

no collector

non-I/O against block I/O

Needs a new postgres.stats.statements collector, after the decision on whether query text may enter the artifact. It also needs track_io_timing on the server.

Most expensive statements

no collector

by total time in the window

Needs a new postgres.stats.statements collector, after the decision on whether query text may enter the artifact.

Logs

Logs are not collected yet

Artifacts do not record this today, so there is nothing to show for this snapshot.